

Profesor: Borja Rey Seoane

TEMA 6

SERVIZOS, MÓDULOS E RUTAS EN ANGULAR

Módulo: **Desenvolvimento Web en Contorna Cliente**

Ciclo: **DAW**

Interfaces

Unha **interface de implementación** (ou simplemente **interface**) é un modelo de código que define a estrutura que deben seguir os obxectos dun certo tipo. Nese «contrato» especifícanse que propiedades, métodos e eventos deberán ser implementados nun obxecto que siga o tal modelo.

As interfaces resultan particularmente útiles para garantir que determinadas clases cumpran co requirimentos específicos, o que facilita a programación orientada a obxectos e aumenta a lexibilidade e a mantibilidade do código.

Tipos de interfaces

Distinguímos dous tipos de interfaces en TypeScript segundo o lugar onde se definan:

- **Interfaces internas:** aquelas definidas dentro do ficheiro doutra clase.

```
interface DetallesProducto {
  descripción: string;
  stock: number;
}

class Producto {
  nome: string;
  prezo: number;

  constructor(nome: string, prezo: number) {
    this.nome = nome;
    this.prezo = prezo;
  }
}
```

- **Interfaces externas:** aquelas definidas nun ficheiro propio.

Creación de interfaces

Nesta sección trataremos as interfaces externas, en tanto que serán as que nos permitan respectar máis fielmente o paradigma da orientación ao obxectos, ao que tempo que axudarán a manter un código máis ordenado e escalable.

Supoñamos pois que queremos definir unha interface para os obxectos de tipo usuario. Coma calquera elemento de clase de Angular dos tratados até o de agora, as interfaces poden ser escritas directamente en ficheiros creados manualmente, aínda que resulta polo xeral máis cómodo empregar a ferramenta `ng` para tales fins. Así, primeiro, executamos o seguinte comando:

```
ng generate interface usuario
```

Ou, na súa variante en *shorthand*:

```
ng g i usuario
```

Unha vez creado o ficheiro procedemos a editalo engadindo as propiedades, métodos e eventos que precisemos (neste caso cinguirémonos a unha estrutura sinxela):

```
export interface Usuario {  
  uid: number;  
  nome: string;  
  login: string;  
  email: string;  
  contrasinal: string;  
}
```

Outro posible exemplo de interface sería unha aplicable aos obxectos de tipo `Pedido`:

```
export interface Pedido {  
  id: number;  
  produto: string;  
  cantidade: number;  
  cliente: string;  
}
```

Propiedades opcionais

As interfaces en TypeScript poden posuír propiedades opcionais, é dicir, cuxa ausencia na declaración dun obxecto non produciría ningún erro. Vexamos un exemplo:

```
export interface Produto {  
  id: number;  
  nome: string;  
  descrición?: string; // Propiedade opcional  
}
```

Propiedades de só lectura

Se queremos establecer unha propiedade como constante, é dicir, que o seu valor non poida ser modificado trala inicialización dos obxectos, empregaremos a palabra reservada `readonly`:

```
export interface Produto {  
  readonly id: number;  
  nome: string;  
}
```

Métodos

Como mencionamos ao inicio do apartado, as interfaces tamén poden definir métodos:

```
// Definición da interface 'Servizo'
export interface Servizo {
  executar(): void;
}

// Utilización da interface 'Servizo' definindo o comportamento do método
'executar'
export class MeuServizo implements Servizo {
  executar(): void {
    console.log('Executando o servizo...');
  }
}
```

Uso de interfaces

Unha vez creada a nosa interface xa podemos empregala para manipular obxectos do tipo correspondente (neste caso, usuarios). Por exemplo, se quixeramos incorporala a unha compoñente, teríamos que importala na súa parte superior, indicando a súa ruta relativa:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
import { Usuario } from './usuario';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'interfaces';
  usuario : Usuario;

  constructor() {
    this.usuario = {
      uid: 1,
      nome: 'Xosé Castro Sandiás',
      login: 'pepe',
      email: 'pepe@meu-correo.gal',
      contrasinal: 'abc123..',
    };
  }
}
```

Outro exemplo de aplicación dunha interface emprega a creada no punto anterior para os obxectos de tipo Pedido:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Pedido } from './pedido.interface';
```

```

@Component({
  selector: 'app-pedido',
  template: `<p>{{ pedido.produto }}</p>`,
})
export class PedidoComponent {
  pedido: Pedido = {
    id: 1,
    produto: 'Libro Angular',
    cantidade: 2,
    cliente: 'Xoán Pérez',
  };
}

```

Exemplo T06E01

Este exemplo introduce o uso de interfaces en TypeScript aplicadas a Angular: defínese a interface `Usuario` con propiedades obrigatorias, opcionais (`departamento?`) e de só lectura (`empresa`), ademais da sinatura dun método. A compoñente crea un obxecto que implementa a interface e emprega un botón para invocar o método `cambiarDepartamento`, actualizando a vista mediante *data binding*.

Obxectivos didácticos do exemplo:

- Comprender que é unha interface en TypeScript e como establece o contrato de forma e comportamento dos obxectos.
- Identificar propiedades obrigatorias, opcionais e `readonly` e os efectos que teñen no deseño do modelo.
- Implementar nun obxecto real os métodos declarados pola interface e empregalos dende a compoñente.
- Relacionar interfaces con enlazado de datos e eventos en Angular para reflectir cambios no estado na vista.

Erros e problemas habituais

Neste apartado veremos algúns dos erros máis habituais que poden xurdir no uso de interfaces de implementación en Angular:

- **Erro: non importar a interface.**
 - Mensaxe típica: "*cannot find name 'Usuario'.*".
 - Solución: verificar que a interface foi importada correctamente.
- **Erro: non cumprir o contrato da interface.**
 - Exemplo: "se a interface define unha propiedade como obrigatoria, a clase ou obxecto deben implementala."
 - Solución: engadir todas as propiedades e métodos definidos na interface.
- **Erro: tipo de dato incorrecto.**
 - Exemplo: "declarar unha propiedade como `string` cando a interface especifica `number`."
 - Solución: verificar os tipos de datos asignados.
- **Erro: confundir clases con interfaces.**
 - Causa: tentar instanciar unha interface directamente.
 - Solución: lembra que unha interface non se pode instanciar. Usa unha clase que implemente a interface para crear obxectos correspondentes con esa interface.

Clases

Unha **clase** tanto en JavaScript coma en TypeScript, e por extensión en Angular, é unha estrutura que fornece unha definición do tipo que terán os obxectos. Coas clases non se traballa directamente, senón que deben ser instanciadas para crear obxectos de xeito que sexan estes os que desenvolvan a lóxica do programa.

Dito doutro xeito, unha clase en Angular é un «molde» para crear obxectos que permite encapsular datos (propiedades) e comportamentos (métodos) nunha única entidade, facilitando a programación orientada a obxectos (POO).

Creación de clases

Do mesmo xeito que ocurría coas clases, estas poden ser creadas manualmente a partires dun ficheiro en branco, pero realizaremos esta acción a través do comando `ng`, por resultar máis práctica:

```
ng generate class usuario
```

Ou tamén, na súa variante curta:

```
ng g cl usuario
```

Unha vez dispoñemos do ficheiro, procederemos a escribir a nosa clase incluíndo todas aquelas propiedades, métodos (xunta con construtores, *getters* e *setters*) e eventos precisos:

```
export class Usuario {
  _uid: number;
  _login: string;

  constructor(uid: number, login: string) {
    this._uid = uid;
    this._login= login;
  }

  mostrarInformacion(): string {
    return ` ${this._login} (${this._uid})`;
  }

  get uid(): number {
    return this._uid;
  }

  get login(): string {
    return this._login;
  }

  set uid(pUid: number): void {
    this._uid = pUid;
  }
}
```

```
set login(pLogin: string): void {  
    this._login = pLogin;  
}  
}
```

Propiedades e métodos

As clases en TypeScript poden conter:

- **Propiedades:** almacenan os datos da clase.
- **Métodos:** executan accións sobre os datos.
- **Construtores:** son un tipo especial de método que inicializa as propiedades cando se crea unha instancia da clase.

Vexamos un exemplo de definición de clase:

```
export class Produto {  
    private prezo: number;  
  
    constructor(public nome: string, prezo: number) {  
        this.prezo = prezo;  
    }  
  
    obterPrezo(): number {  
        return this.prezo;  
    }  
  
    actualizarPrezo(novoPrezo: number): void {  
        if (novoPrezo > 0) this.prezo = novoPrezo;  
    }  
}
```

Getters e setters

Son métodos especiais predefinidos no paradigma da POO que permiten acceder (*getters*) e modificar (*setters*) propiedades privadas dunha forma controlada:

```
export class Vehiculo {  
    constructor(private _marca: string, private _modelo: string) {}  
  
    get marca(): string {  
        return this._marca;  
    }  
  
    set marca(novaMarca: string) {  
        if (novaMarca) this._marca = novaMarca;  
    }  
}
```

Uso de clases

Coa nova clase creada, procedemos a empregala dende a nosa aplicación, por exemplo, dende a compoñente raíz. Co gallo diso, importámola na súa parte superior, indicando a ruta relativa do ficheiro que contén a definición da clase:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
import { Usuario } from './usuario;

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'interfaces';
  usuario : Usuario;

  constructor() {
    this.usuario = new Usuario(1, 'lola');
  }
}
```

Exemplo T06E02

Neste exemplo defínense as clases Grupo e Usuario con construtores e acceso ás propiedades mediante *getters/setters*. Usuario compón unha listaxe de Grupo e a compoñente instancia obxectos, amosa propiedades por interpolación e percorre as membresías cun **ngFor*.

Este exemplos persegue os seguintes obxectivos didácticos:

- Comprender a definición de clases e o uso de construtores en TypeScript.
- Diferenciar estado interno (propiedades privadas convencionais) e acceso controlado con *getters/setters*.
- Aplicar composición para modelar relacións un-a-moitos entre obxectos (Usuario → Grupo[]).
- Crear instancias dende a compoñente e inicializar o modelo con datos.
- Empregar enlazado de datos e **ngFor* para representar coleccións na vista.

Heranza en clases

TypeScript permite tamén que unha clase herde propiedades e métodos doutra:

```
export class Vehiculo {
  constructor(public marca: string, public modelo: string) {}

  mostrarDatos(): string {
    return `${this.marca} ${this.modelo}`;
  }
}

export class Coche extends Vehiculo {
  constructor(marca: string, modelo: string, public portas: number) {
    super(marca, modelo);
  }

  mostrarDatos(): string {
    return `${super.mostrarDatos()}, ${this.portas} portas`;
  }
}
```

Erros e problemas habituais

Neste apartado veremos algúns dos erros máis habituais que poden xurdir no uso de clases en Angular:

- **Erro: non declarar propiedades ou métodos no construtor.**
 - Mensaxe: *"cannot find name 'propiedade'."*
 - Solución: declare as propiedades de forma explícita.
- **Erro: modificación de propiedades privadas.**
 - Causa: intentar acceder a unha propiedade privada directamente.
 - Solución: use métodos ou `setters/getters` para modificar propiedades privadas.
- **Erro: non inicializar todas as propiedades.**
 - Mensaxe: *"property 'propiedade' has no initializer and is not definitely assigned in the constructor."*
 - Solución: empregue un inicializador ou o modificador `!` para indicar que a propiedade será asignada máis tarde.
- **Erro: problemas coa heranza.**
 - Causa: omitir o uso do método `super()` no construtor dunha clase derivada.
 - Solución: asegúrese de chamar a `super()` coas propiedades necesarias.

Módulos

Os **módulos** en Angular son clases que permiten organizar a funcionalidade da nosa aplicación. Actúan coma contedores doutros elementos (compoñentes, directivas, servizos e outras clases) relativos a unha mesma finalidade ou funcionalidade común. Dito doutro xeito, os módulos son contedores de alto nivel para un conxunto de capacidades da aplicación á cal pertencen.

Do mesmo xeito que ocorre coas directivas, existe toda unha librería de módulos nativos propios de Angular que podemos empregar, así coma podemos crear os nosos propios módulos se así o precisamos.

Dado que dende Angular 17 o modo de traballo por defecto é *standalone*, isto é, sen módulos, se queremos traballar con módulos primeiramente precisamos crear os nosos proxectos engadindo o modificador correspondente ao comando:

```
ng new modulos --no-standalone
```

Conceptos fundamentais

Os módulos en Angular ofrecen diversos beneficios no desenvolvemento de aplicacións:

- **Encapsulamento:** cada módulo serve como unha fronteira lóxica, permitindo encapsular unha funcionalidade específica. Isto axuda a manter o código máis organizado (tal e como establece o paradigma tradicional da programación modular).
- **Reemprega:** os módulos poden ser facilmente reempregados en diferentes partes da aplicación ou mesmo en distintas aplicacións.
- **Mantemento:** o código faise máis doado de manexar e actualizar. Os cambios realizados sobre un módulo teñen menos probabilidade de afectar a outros aspectos da aplicación.
- **Carga preguiceira¹:** Angular permite cargar módulos de maneira asíncrona, o que pode mellorar notablemente o rendemento da aplicación (en tanto cárganse só os módulos que son precisos en cada intre).

Angular distingue tamén os módulos en dous tipos:

- O **módulo raíz** (`AppModule`), creado de xeito automático nas aplicacións de tipo non *standalone* (ou modulares). O seu cometido é integrar as diferentes partes da aplicación e, polo xeral tamén, albergar a lóxica de arranque.

¹ Do inglés *lazy loading*.

- Os **módulos funcionais** ou **de característica**, aqueles creados para agrupar funcionalidades específicas. Por exemplo, un módulo para xestionar usuarios, outro para manexar pedidos etc. Estes módulos axudan a manter a funcionalidade da aplicación separada e organizada.

Estrutura dun módulo

Un módulo en Angular tipicamente contén catro partes principais:

- **Declaracións (declarations):** lista as compoñentes, directivas e tubaxes que pertencen ao módulo. Estes elementos son privados por defecto e só serán accesibles dende dentro do módulo, non sendo que sexan exportados.
- **Importacións (imports):** importa outros módulos e as súas funcionalidades. Por exemplo, se o teu módulo precisa usar formularios ou HTTP, deberás importar `FormsModule` e `HttpClientModule` respectivamente.
- **Provedores (providers):** lista os servizos que este módulo contribúe á aplicación. Estes servizos poden ser inxectados en compoñentes e outros servizos dentro do módulo.
- **Exportacións (exports):** aquí podes exportar compoñentes, directivas e tubaxes (*pipes*) para que outros módulos poidan empregarlas.
- **Arrinque (bootstrap):** especifica a compoñente raíz no módulo principal.

Tipos de módulos

Veremos a continuación os distintos tipos de módulos que ten Angular segundo para que serven e como se usan:

- **Módulo Raíz (AppModule):** é o punto de entrada da aplicación, onde se configuran as compoñentes principais e se define o arranque da aplicación.

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, AppRoutingModule],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

- **Módulos Funcionais:** estes módulos encapsulan unha funcionalidade específica (como, por exemplo, usuarios ou produtos). Son ideais para dividir aplicacións grandes.

```
@NgModule({
  declarations: [UsuarioComponent],
  imports: [CommonModule],
  exports: [UsuarioComponent]
})
export class UsuarioModule {}
```

- **Módulos Compartidos:** usados para compoñentes, directivas e *pipes* que deben ser reutilizables noutros módulos.

```
@NgModule({
  declarations: [CabeceraComponent, RodapeComponent],
  exports: [CabeceraComponent, RodapeComponent],
  imports: [CommonModule]
})
export class SharedModule {}
```

- **Módulos con *lazy loading*:** permiten cargar módulos baixo demanda, mellorando o rendemento da aplicación.

```
const rutas: Routes = [
  { path: 'usuarios', loadChildren: () =>
import('./usuario/usuario.module').then(m => m.UsuarioModule) }
];
@NgModule({
  imports: [RouterModule.forRoot(rutas)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

Creación de módulos

Supoñamos polo tanto que partimos da base de dispoñer dun proxecto modular novo recién creado (ao que chamamos **módulos**).

Para crear módulos de Angular empregamos o seguinte comando:

```
ng generate module novo
```

Ou na súa forma reducida:

```
ng g m novo
```

Ao executar calquera dos comandos anteriores créase, de xeito automático, dentro do directorio `src/app` un directorio de nome `novo` que conterá o ficheiro do novo módulo (`novo.module.ts`).

Exemplo T06E03

Neste exemplo defínese un módulo funcional que declara unha compoñente de lista, consume un servizo inxectable con `providedIn: 'root'` para obter datos de usuario e exporta a compoñente para poder usala noutros ámbitos. A vista amosa a colección mediante `*ngFor`.

Obxectivos didácticos:

- Comprender a finalidade dun `NgModule`: declarar, importar e exportar elementos da aplicación.
- Entender como un servizo con `providedIn: 'root'` ofrece unha instancia única e pode ser consumido por calquera compoñente do proxecto.
- Separar responsabilidades: o servizo fornece datos e a compoñente encárgase da presentación.
- Empregar `*ngFor` para renderizar listas a partir de datos do servizo.
- Aprender a expor compoñentes dun módulo para seren reutilizadas noutros ámbitos da aplicación.

Carga preguiceira

Xa introducida anteriormente, a **carga preguiceira** ou **lazy loading** é unha técnica en Angular que permite cargar módulos só cando son necesarios, o que axuda a reducir o tempo de carga inicial da aplicación. Vexamos como implementala nas nosas aplicacións paso a paso:

- Crear un módulo que desexemos cargar de xeito asíncrono.
- Configura as rutas no módulo raíz ou noutro módulo de enrutamento, definindo unha ruta para o módulo con carga preguiceira empregando a propiedade `loadChildren`. Por exemplo:

```
{
  path: 'nomeDaRuta', loadChildren: () =>
    import('./nome-do-modulo/nome-do-modulo.module').then(m =>
      m.NomeDoModuloModule)
}
```

- A partires dese punto, cando un usuario navegue á ruta especificada, Angular cargará o módulo correspondente de maneira asíncrona.

Exemplo T06E04

Neste exemplo configúrase unha ruta que, ao navegar cara a un segmento concreto, carga de forma diferida un módulo de funcionalidade mediante `loadChildren` con importación dinámica. O módulo só se descarga e inicializa cando se visita esa ruta, reducindo o peso inicial da aplicación e mellorando o tempo de arranque.

Obxectivos didácticos deste exemplo:

- Comprender o patrón de carga diferida e cando resulta útil para optimizar a aplicación.
- Entender a configuración dunha ruta con `loadChildren` usando unha importación dinámica e a resolución do módulo destino.

- Ver como a navegación a unha ruta concreta desencadea a descarga e inicialización do módulo asociado.
- Distinguir entre a configuración principal de rutas e a configuración interna do módulo cargado de forma diferida.
- Relacionar *lazy loading* con melloras de rendemento: división de código, menor *bundle* inicial e carga baixo demanda.

Aplicacións modulares

Neste punto aprenderemos a crear unha aplicación Angular básica que conste de dous módulos funcionais: un módulo de usuarios e outro de produtos.

Partindo da base de que teñamos unha aplicación modular (non *standalone*) creada, procedemos crear dous novos módulos:

```
ng generate module usuarios
ng generate module produtos
```

Módulo de usuarios

Es módulo xestionará todo o relacionado cos usuarios, con ese fin:

- Creamos unha compoñente para listar usuarios:

```
ng generate component usuarios/listarUsuarios
```

- E outra compoñente para editar usuarios:

```
ng generate component usuarios/editarUsuario
```

- Engadimos as funcionalidades precisas para incorporar datos de usuarios (formularios, CRUD etc.).

Módulo de produtos

De xeito análogo ao anterior, este módulo conterá a funcionalidade relativa aos produtos, para o que tomaremos os seguintes pasos:

- Creamos unha compoñente para listar produtos:

```
ng generate component produtos/listaProdutos
```

- E creamos tamén unha compoñente para ver o detalle dos produtos:

```
ng generate component produtos/detallesProducto
```

- Engadimos as funcionalidades precisas para a xestión dos produtos (obtención de detalles, listaxe etc).

Integración dos módulos

O seguinte paso será integrar os novos módulos na aplicación mediante a súa importación no módulo raíz (AppModule). Con ese fin, no ficheiro app.module.ts engadimos as seguintes liñas no seu inicio:

```
import { UsuariosModule } from './usuarios/usuarios.module';  
import { ProductosModule } from './productos/productos.module';
```

Se ademais estamos a desenvolver unha aplicación que faga uso de rutas, é preciso definir as rutas que dean acceso ás compoñentes dos distintos módulos. Con ese fin definiríamos, en AppRoutingModuleModule, as seguintes rutas:

```
const routes: Routes = [  
  {  
    path: 'usuarios', loadChildren: () =>  
      import('./usuarios/usuarios.module').then(m =>  
        m.UsuariosModule)  
  },  
  {  
    path: 'productos', loadChildren: () =>  
      import('./productos/productos.module').then(m =>  
        m.ProductosModule)  
  }  
];
```

Finalmente, en cada módulo empregariamos RouterModule para definir as rutas específicas de cada compoñente.

Comunicación entre módulos

A comunicación entre módulos de Angular pódese realizar mediante paso de parámetros nas rutas, pero o método máis habitual (e seguro) na actualidade é a emprega de servizos globais que son inxectados nas distintas compoñentes dos módulos.

Este concepto tratarémolo máis adiante no capítulo seguinte, polo que non nos estenderemos máis aquí no mesmo.

Erros e problemas habituais

Neste apartado veremos algúns dos erros máis habituais que poden xurdir no uso de módulos en Angular:

- **Erro: Non exportar compoñentes ou módulos.**
 - Mensaxe: *"component is not part of any NgModule."*
 - Causa: non se incluíu o compoñente na sección `exports`.
 - Solución: engadir o compoñente á lista de exportacións do módulo correspondente.
- **Erro: Problemas coa carga preguiceira.**
 - Mensaxe: *"cannot find module '...'"*
 - Causa: erro ao definir a ruta de carga do módulo.
 - Solución: comprobar que a ruta no `loadChildren` apunta ao ficheiro correcto.
- **Erro: Duplicación de servizos.**
 - Causa: declarar un servizo en máis dun módulo.
 - Solución: usar `providedIn: 'root'` para evitar a duplicación.

Servizos

Os **servizos** constitúen un dos elementos básicos dos proxectos de Angular, en tanto que permiten alixeirar ás compoñentes de lóxica ao tempo que melloramos a organización do noso código, o que redunda na escalabilidade xeral do proxecto. Ademais do anterior, os servizos actúan coma repositorios de funcionalidade (ao xeito de librarías), permitindo a compoñentes distintas facer uso do mesmo código sen necesidade de replicalo en cada unha delas. Na práctica o anterior tradúcese en que os servizos permiten a comunicación directa e efectiva entre compoñentes.

Creación de servizos

No tempo de crear un servizo, e de xeito semellante a coma faciamos para crear compoñentes, empregamos o seguinte comando:

```
ng generate service servizo
```

Ou na súa variante curta:

```
ng g s servizo
```

Así mesmo, e tal e coma ocorría coas compoñentes, podemos aproveitar no intre da creación para organizar a estrutura dun proxecto modular introducindo o novo servizo dentro dun módulo concreto:

```
ng g s modulo/servizo
```

Inxectar e empregar servizos

Vexamos agora o procedemento de integración dun novo servizo (**Servizo**) dentro da compoñente raíz (**app.component**) do noso proxecto *standalone*:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
import { ServizoService } from './servizo.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  providers: [ ServizoService ]
})
export class AppComponent {
  title = 'Servizos Standalone';
  texto : string;
```

```
// Ao incorporar o servizo como parámetro privado no constructor estamos a
// tornalo nunha propiedade de clase privada de xeito automatizado
constructor(private servizo: ServizoService) {
  this.texto = servizo.texto;
}
}
```

Como se pode apreciar nas liñas marcadas en negra, para engadir un servizo a unha compoñente *standalone* é preciso engadilo ao seu *array providers* no decorador (ademais de incorporar a correspondente importación na parte superior).

NOTA IMPORTANTE: os servizos só son importados no *array providers* na compoñente de nivel máis alto da xerarquía do conxunto de compoñentes que vaian usalo. Así, se nos temos unha compoñente nai con dúas compoñentes fillas, e todas elas fosen empregar o servizo, importariamos o mesmo en cada unha delas, pero só o colocaríamos no *array providers* da compoñente nai.

Tipos de servizos

Segundo como se inxecten nas compoñentes que os usarán, Angular distingue dous tipos de servizos:

- **Servizos *singleton*:** inclúense na propiedade `providedIn: 'root'`. Son instancias únicas compartidas en toda a aplicación.
- **Servizos específicos:** declarados na propiedade `providers` dun módulo ou compoñente. A súa instancia é limitada ao contorno onde foron declarados.

Implementando servizos

Os servizos, a diferenza das compoñentes, non son elementos de vista, polo que non dispoñen de ficheiros **html** nin **css**, senón que constan unicamente de código **TypeScript**.

A continuación vemos un exemplo co código do noso servizo **NovoServizo** (no ficheiro **novo-servizo.service.ts**):

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})

export class ServizoService {
  texto = 'Ola mundo (dende o novo servizo)!';

  constructor() {}
}
```

Empregando servizos

Cando queremos empregar un servizo da nosa creación dende unha compoñente unicamente temos que crealo coma se dunha propiedade calquera se tratara. Vexamos como quedaría o código da nosa compoñente raíz (**app.component.ts**) se quixeramos empregar **ServizoService**:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
import { ServizoService } from '../servizo.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'servizos';
  texto : string;

  constructor(servizo: ServizoService) {
    this.texto = servizo.texto;
  }
}
```

Agora xa poderíamos acceder a calquera das propiedades e métodos do noso servizo xa sexa dende a vista (o HTML) ou o controlador (TS) da nosa compoñente. Por exemplo, no caso de **app.component.html** sería así (lembremos que a propiedade `texto` toma o valor da propiedade homónima no servizo:

```
<p>{{ texto }}</p>
```

Exemplo T06E05

Neste exemplo defínese un servizo inxectable no ámbito global da aplicación e dúas compoñentes *standalone* que o consumen. Ambas acceden ao mesmo dato exposto polo servizo, ilustrando o patrón *singleton* cando se usa `providedIn: 'root'`.

Obxectivos didácticos deste exemplo:

- Comprender a creación dun servizo con `@Injectable` e o seu ámbito mediante `providedIn: 'root'`.
- Aplicar a inxección de dependencias en compoñentes *standalone* a través do construtor.
- Identificar o comportamento *singleton* dun servizo global e o compartido de estado/datos entre compoñentes.
- Valorar boas prácticas ao expoñer datos en servizos (lectura, mutación controlada, encapsulamento).

Erros e problemas habituais

Neste apartado veremos algúns dos erros máis habituais que poden xurdir no uso de servizos en Angular:

- **Erro: "no provider for [Servizo]".**
 - Solución: asegurarse de declarar o servizo na propiedade `providers` ou usar `providedIn: 'root'`.
- **Erro: non inxectar o servizo correctamente ("cannot read property of undefined"):**
 - Causa: non empregar o modificador `private` ou `public` no constructor.
 - Solución: usar a sintaxe correcta para a inxección:

```
constructor(private servizo: NomeServizo) {}
```

- **Erro: dobre instancia do servizo:**
 - Causa: Declarar o servizo en múltiples módulos.
 - Solución: Usar `providedIn: 'root'` ou importar o servizo no módulo raíz.
- **Erro ao inxectar HttpClient:**
 - Causa: non importar `HttpClientModule`.
 - Solución: engadir `HttpClientModule` ao array `imports` do módulo raíz:

```
import { HttpClientModule } from '@angular/common/http';
```

- **Erro: uso incorrecto do ciclo de vida:**
 - Causa: tentar usar o servizo antes de inicializar a compoñente.
 - Solución: empregar `ngOnInit` para as operacións asíncronicas.

Observables

Os **observables** en Angular son fontes de datos que veñen definidas no paquete RxJS². Ese paquete, desenvolvido por Microsoft para JS, pero empregable baixo TS, posibilita a **programación reactiva**³ (son polo tanto unha alternativa ás promesas).

Os observables son obxectos que seguen o patrón de deseño homónimo, e fundaméntanse nos seguintes conceptos:

- **Observable:** o emisor de datos.
- **Observador:** o receptor ou subscritor que consome os datos emitidos.
- **Subscrición:** a conexión entre o observable e o observador.
- **Operadores:** funcións que transforman ou manipulan os datos emitidos polos observables.

Funcionalmente, os observables permiten manexar tarefas asíncronas (isto é, que non sabemos cando son lanzadas nin canto durarán). Así, cando a funcionalidade do noso proxecto inclúa tarefas ou accións cuxa finalización non sexa estritamente necesario coñecer pola nosa parte, empregaremos observables para manexalas.

O manexo de Observables en Angular presenta unha serie de vantaxes:

- Só se executan se teñen subscritores.
- Encárganse do manexo de erros de xeito autónomo.
- Fornecen moita flexibilidade no referente a manipulación de datos.
- Permiten desuscribirse en calquera intre, o que os cancela automaticamente (isto redunda nun mellor uso da memoria no lado do cliente).

Instalando as dependencias

Dado que os **observables** pertencen a un paquete de terceiros, para a súa instalación empréganse certos parámetros de npm que non atopamos nos paquetes propios do *framework*. Así executaremos a seguinte orde, que instalará o propio paquete de RxJS:

```
npm install rxjs
```

Empregando observables

Coma en casos anteriores, crearemos un novo proxecto ao cal engadiremos:

² *Reactive Extensions Library for JavaScript*.

³ Paradigma de desenvolvemento que pon o foco no manexo asíncrono de fluxos de datos.

- Dúas compoñentes: **formulario** e **listaxe**.
- Un servizo: **modulos**.
- Unha interface: **modulo**.

A compoñente raíz

Neste primeiro caso, imos ao HTML da compoñente raíz (**app.component.html**) e engadimos as etiquetas de ámbalas dúas compoñentes:

```
<app-formulario></app-formulario>
<app-listaxe></app-listaxe>
```

O seu ficheiro de controlador (**app.component.ts**) ficaría como segue:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
import { FormularioComponent } from '../formulario/formulario.component';
import { ListaxeComponent } from '../listaxe/listaxe.component';
import { ModulosService } from '../modulos.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, FormularioComponent, ListaxeComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  providers: [ModulosService]
})

export class AppComponent {
  title = 'observables';
}
```

A compoñente formulario

No ficheiro **formulario.component.ts** poñemos o seguinte:

```
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';
import { ModulosService } from '../modulos.service';

@Component({
  selector: 'app-formulario',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './formulario.component.html',
  styleUrls: ['./formulario.component.css']
})

export class FormularioComponent {
  formulario: FormGroup;
```

```

    constructor(private servizoModulos: ModulosService) {
        this.formulario = new FormGroup({
            nome: new FormControl(),
            horas: new FormControl(),
            curso: new FormControl(),
        });
    }

    onSubmit() {
        this.servizoModulos.engadirModulo(this.formulario.value);
    }
}

```

Definimos tamén na súa vista (**formulario.component.html**) o código:

```

<form [formGroup]="formulario" (ngSubmit)="onSubmit()">
  <div class="form-group">
    <label>Nome</label>
    <input type="text" class="form-control" formControlName="nome">
  </div>
  <div class="form-group">
    <label>Horas</label>
    <input type="number" class="form-control" formControlName="horas">
  </div>
  <div class="form-group">
    <label>Curso</label>
    <input type="number" class="form-control" formControlName="curso">
  </div>
  <input type="submit" value="Crear">
</form>

```

A compoñente listaxe

Esta compoñente amosará a listaxe de módulos dados de alta. Para iso, incluiremos no ficheiro **listaxe.component.ts** o seguinte contido:

```

import { Component, OnInit } from '@angular/core';
import { ModulosService } from '../modulos.service';
import { Modulo } from '../modulo';
import { CommonModule } from '@angular/common';

@Component({
    selector: 'app-listaxe',
    standalone: true,
    imports: [CommonModule],
    templateUrl: './listaxe.component.html',
    styleUrls: ['./listaxe.component.css']
})

export class ListaxeComponent implements OnInit {
    listaxeModulos: Modulo[];

    constructor(private servizoModulos: ModulosService) {}

    ngOnInit(): void {
        // Subscribímonos aos cambios que se produzan no servizo
        this.servizoModulos.getModulos$.subscribe((modulos) => {
            this.listaxeModulos = modulos;
        });
    }
}

```

E no seu ficheiro HTML (**listaxe.component.html**) isto:

```
<div class="modulos">
  <div class="modulo" *ngFor="let modulo of listaxeModulos">
    <ul>
      <li>Nome: {{modulo.nome}}</li>
      <li>Duración: {{modulo.horas}} horas</li>
      <li>Curso: {{modulo.curso}}°</li>
    </ul>
  </div>
</div>
```

O servizo modulos

Definiremos agora o comportamento do noso servizo no seu ficheiro **modulos.service.ts** do xeito que vemos a continuación:

```
import { Injectable } from '@angular/core';
import { Modulo } from './modulo';
import { Subject, Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})

export class ModulosService {
  // Definimos un array que conterá os módulos que se vaian creando.
  private modulos: Modulo[];
  // Definimos un subject que emitirá o array de módulos para todos os elementos
  // que estean subscritos a el.
  private modulos$: Subject<Modulo[]>;

  constructor() {
    this.modulos = [];
    this.modulos$ = new Subject();
  }

  engadirModulo(pModulo: Modulo): void {
    this.modulos.push(pModulo); // Engadimos o novo módulo ao array
    this.modulos$.next(this.modulos); // Enviamos a todos os subscritos o array
    // actualizado
  }

  getModulos$(): Observable<Modulo[]> {
    return this.modulos$.asObservable(); // Devolvemos o Subject como Observable
    // para que outros elementos poidan subscribirse ao mesmo.
  }
}
```

A interface módulo

Velaquí a interface de implementación que empregamos para definir os obxectos de tipo «Módulo»:

```
// Definimos unha interface que nos dea a estrutura dos obxectos de tipo 'módulo'
por comodidade.
export interface Modulo {
  nome: string;
  horas: number;
  curso: number;
}
```

Exemplo T06E06

Neste exemplo o servizo expón un estado compartido como fluxo observable mediante un `BehaviorSubject`. Dúas compoñentes se subscriben a ese fluxo para reflectir en tempo real os cambios do texto, e tamén poden emitir actualizacións chamando a un método do servizo que fai `next` no `subject`.

Os obxectivos didácticos deste exemplo son:

- Comprender o patrón de estado compartido reactivo nun servizo usando `BehaviorSubject`.
- Diferenciar entre expor o `subject` como `Observable` (`asObservable`) e actualizar o estado co método público do servizo.
- Aplicar subscricións en `ngOnInit` para recibir actualizacións e chamar a métodos do servizo para publicar cambios.
- Introducir boas prácticas básicas de RxJS en compoñentes (subscrición explícita e responsabilidade do servizo como única fonte de verdade).

Exemplo T06E07

Neste exemplo un servizo mantén en memoria unha listaxe de incidencias e expóna como fluxo observable mediante un `BehaviorSubject`. Un formulario reactivo crea novas incidencias e publícaa no servizo; unha compoñente da listaxe subscríbese ao observable e actualiza a visualización en tempo real. A instancia do servizo declárase como provedor a nivel da compoñente raíz para compartir o mesmo estado entre as compoñentes implicadas.

Este exemplo persegue os seguintes obxectivos didácticos:

- Comprender o patrón publicador–subscritor con `BehaviorSubject` aplicado a coleccións.
- Integrar formularios reactivos para xerar datos e comunicalos ao servizo.
- Actualizar a interface en tempo real mediante subscricións ao observable do servizo.

- Distinguir o ámbito de inxección do servizo cando se fornece a nivel de raíz da aplicación fronte a provedores locais.

Erros e problemas habituais

Neste apartado veremos algúns dos erros máis habituais que poden xurdir no uso de observables en Angular:

- **Erro: non subscribirse a un observable.**
 - Causa: os observables non se executan sen unha subscrición.
 - Solución: asegura que `.subscribe()` é chamado.
- **Erro: fuga de memoria debido a subscricións non canceladas.**
 - Solución: desubscríbete manualmente ou emprega operadores como `takeUntil`.
- **Erro ao combinar observables con diferentes tipos de datos.**
 - Solución: asegura que os tipos son compatibles antes de empregar operadores.
- **Erro: non manexar erros nos observables.**
 - Solución: usa o bloque error dentro de `.subscribe()` ou operadores como `catchError`.

Animacións

Coma xa sabemos, en programación Web as animacións fornecen a ilusión do movemento a través da mudanza dos estilos dos elementos HTML ao longo do tempo.

Un correcto uso das animacións pode facer as nosas aplicacións máis usables, accesibles, interactivas e intuitivas, resultando nunha mellora xeral da experiencia de usuario.

O mecanismo de animacións integrado en Angular basea a súa funcionalidade en CSS, respectando o estándar definido pola W3C.

As **animacións** en Angular permiten mellorar a experiencia do usuario ao proporcionar un comportamento visual máis interactivo e intuitivo. Empregan unha combinación de **CSS** e **TypeScript**, aproveitando o motor de renderizado de Angular para realizar transicións, transformacións e cambios de estado fluídos.

Instalando as dependencias

Para empregar o módulo de animacións nativo de Angular é preciso instalalo a través do xestor de paquetes de NodeJS, do xeito seguinte:

```
npm install @angular/animations
```

Empregando animacións

Coma ocorre co resto de módulos do *framework*, para empregar o módulo de animacións é preciso importalo dende aqueles lugares (módulos e compoñentes) que vaian empregalo. No seguinte exemplo traballaremos nun proxecto modular novo (non *standalone*), sen compoñentes, servizos nin módulos adicionais, polo que empregaremos simplemente o módulo e a compoñente raíces.

Comezaremos por abrir o ficheiro correspondente ao módulo raíz (**app.module.ts**), ao que engadiremos as importacións correspondente (en negra figuran as liñas engadidas):

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { BrowserAnimationsModule } from '@angular/platform-browser/animations'
```

```

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    BrowserAnimationsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})

export class AppModule { }

```

De xeito auxiliar imos crear un ficheiro auxiliar (**datos.ts**) que conterá datos útiles para a xeración de animacións; co contido que figura a continuación:

```

// Definimos un array de obxectos.
const datos = [
  {
    nome: 'Desenvolvemento Web en Contorno Cliente',
  },
  {
    nome: 'Desenvolvemento Web en Contorno Servidor',
  },
  {
    nome: 'Deseño de Interfaces Web',
  },
  {
    nome: 'Despregamento de Aplicacións Web',
  },
  {
    nome: 'Empresa e Iniciativa Emprendedora',
  },
];

// Exportamos o array de obxectos para poder empregalo dende o resto do proxecto.
export default datos;

```

Procedemos a continuación a modificar o ficheiro TypeScript da nosa compoñente raíz (**app.component.ts**), o cal ficaría do xeito:

```

import { Component } from '@angular/core';
import { trigger, style, transition, animate, state } from '@angular/animations';
import datos from './datos';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  animations: [
    /* Os triggers determinan o desencadeante dunha animación en base a unha
    listaxe de estados e transicións */
    trigger('estadoInicial', [
      /* O estado baleiro é no que se atopa un elemento que non figura aínda na
      vista (ou que foi eliminado dela) */
      state('void', style({
        transform: 'translateX(-100%)',
        opacity: 0
      })),
    ]),
  ],
})

```

```

        transition(':enter', [
            animate(1000, style({
                transform: 'translateX(0)',
                opacity: 1
            })))
        ])
    ])
}
})

export class AppComponent {
    title = 'animacions';
    public modulos: any[] = datos;
}

```

Así, engadimos o *array* `animations` ao decorador da compoñente, coas seguintes características:

- `trigger`: este método permítenos indicar cal é o desencadeante da animación.
 - O primeiro parámetro recibido é o nome que asignamos á animación (neste caso `estadoIncial`).
 - O segundo parámetro é un *array* de todos os pasos polos que transcorrerá a nosa animación (xunta cos seus estados iniciais).
- `state`: define o estado da animación.
 - `void`: implica que o elemento aínda non foi engadido ao DOM (é dicir, aínda non cargou na páxina).
- `transition`: este método permite indicar cal será o seguinte paso dunha animación.
 - `:enter`: implica que o elemento acaba de ser engadido ao DOM (é dicir, xa cargou na páxina).
 - `animate`: este método indica o tempo de retardo (ou adianto) da animación xunta co estilo asociado á mesma.
- `style`: este método permite definir o estilo CSS a aplicar.

E na súa vista HTML, isto é, en `app.component.html` escribimos:

```

<div *ngFor="let modulo of modulos" @estadoIncial class="tarxeta">
  <h2>{{modulo.nome}}</h2>
</div>

```

E no ficheiro xeral de estilos do proxecto (`styles.css`) poñemos isto:

```

/* You can add global styles to this file, and also import other style files */
body {
    font-family: Robot, Helvetica, Arial;
    background-color: bisque;
}

```

```
.tarxeta {
  border-radius: 4px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.12), 0 1px 2px rgba(0,0,0,0.24);
  box-sizing: border-box;
  padding: 10px 15px;
  margin: 1em;
  background-color: white;
  width: 600px;
}
```

Vexamos aquí outro exemplo práctico de aplicación das animacións en Angular:

```
import { trigger, state, style, transition, animate } from '@angular/animations';

@Component({
  selector: 'app-animacion',
  template: `<div [@cambiarEstado]="estado" (click)="alternarEstado()">Clic para
cambiar estado</div>`,
  styles: [`div { width: 200px; height: 100px; background-color: lightblue; }`],
  animations: [
    trigger('cambiarEstado', [
      state('inicial', style({ transform: 'scale(1)', backgroundColor: 'lightblue'
}))),
      state('final', style({ transform: 'scale(1.5)', backgroundColor: 'blue' })),
      transition('inicial <=> final', animate('300ms ease-in-out')),
    ]),
  ],
})
export class AnimacionComponent {
  estado = 'inicial';

  alternarEstado() {
    this.estado = this.estado === 'inicial' ? 'final' : 'inicial';
  }
}
```

Exemplo práctico: listaxe animada

Imos crear unha animación que amose elementos dunha listaxe de xeito progresivo:

- **Configuración inicial:** definir un *array* para os elementos.
- **Animación de entrada:** empregar `:enter` para animar novos elementos.

```
import { Component } from '@angular/core';
import { trigger, style, transition, animate, query, stagger } from
'@angular/animations';

@Component({
  selector: 'app-lista-animada',
  template: `
<button (click)="engadirElemento()">Engadir elemento</button>
<div class="lista">
  <div *ngFor="let item of elementos" @entrada>
    {{ item }}
  </div>
</div>
`,
  styles: [`
.lista div { margin: 5px 0; padding: 10px; background: lightgreen; border-
radius: 5px; }
`],
})
```

```

    animations: [
      trigger('entrada', [
        transition(':enter', [
          style({ opacity: 0, transform: 'translateY(-20px)' }),
          animate('500ms ease-out', style({ opacity: 1, transform: 'translateY(0)'
        })),
      ]),
    ]),
  ],
})
export class ListaAnimadaComponent {
  elementos: string[] = [];

  engadirElemento() {
    this.elementos.push(`Elemento ${this.elementos.length + 1}`);
  }
}

```

Exemplo T06E08

Este exemplo introduce as animacións declarativas de Angular nunha lista de tarefas: defínense dous *triggers*, un para transicións de aparición e eliminación con `:enter/:leave` e *keyframes*, e outro baseado en estados (*pendente/completada*) que modifica estilo e escala ao marcar unha tarefa. As accións de engadir, completar e eliminar disparan automaticamente as animacións correspondentes.

Obxectivos didácticos do exemplo:

- Comprender a estrutura básica dunha animación con `trigger`, `transition`, `style` e `animate`.
- Aplicar animacións de entrada e saída en elementos renderizados dinamicamente.
- Empregar estados de animación con `state` e transicións bidireccionais.
- Integrar animacións co fluxo de datos e eventos da vista (engadir, completar, eliminar).

Exemplo T06E09

Este exemplo implementa un menú lateral que aparece e desaparece dende a marxe dereita empregando animacións con estados. Defínense dous estados, *oculto* e *visible*, que modifican `transform` e `opacity`, cunha transición bidireccional suavizada. A visibilidade contrólase cun booleano ligado ao método `alternarMenu`, que cambia o estado da animación.

A través deste exemplo procuramos os obxectivos didácticos:

- Entender o uso de `trigger`, `state`, `style`, `transition` e `animate` para crear animacións baseadas en estados.
- Aplicar transformacións CSS (`translateX`) e opacidade para animacións de entrada e saída lateral.

- Ligar o estado da interface a propiedades da compoñente para activar animacións de forma declarativa.
- Crear menús deslizantes accesibles e reutilizables con configuración mínima.

Exemplo T06E10

Este exemplo crea un botón con dous estados visuais (*normal/premido*) controlados por animacións declarativas. Modifícanse `transform`, `backgroundColor` e `color` segundo o estado, cunha transición bidireccional suave activada ao premer o botón.

Obxectivos didácticos procurados mediante este exemplo:

- Empregar `trigger`, `state`, `style`, `transition` e `animate` para definir animacións baseadas en estados.
- Vincular o estado da animación a unha propiedade da compoñente e cambiala mediante eventos.
- Comprender como combinar transformacións e estilos para *feedback* visual inmediato ao usuario.
- Construír un patrón reusable de botón animado con configuración mínima.

Animacións compostas

Para animar múltiples elementos simultaneamente, empregamos `query` e `stagger`:

```
trigger('listaAnimada', [
  transition('* => *', [
    query(':enter', [
      style({ opacity: 0, transform: 'translateY(-20px)' }),
      stagger('100ms', [
        animate('300ms ease-out', style({ opacity: 1, transform: 'translateY(0)'
      ])),
    ]),
  ], { optional: true }),
]),
]);
```

Erros e problemas habituais

Neste apartado veremos algúns dos erros máis habituais que poden xurdir no uso de animacións en Angular:

- **Erro: esquecer importar `BrowserAnimationsModule`:**
 - Mensaxe: *no provider for AnimationBuilder*.
 - Solución: verifica que o módulo está importado en `AppModule`.

- **Erro: confusión entre `animate` e `transition`:**
 - Problema: empregar `animate` fóra dun `transition`.
 - Solución: asegúrate de definir transicións adecuadas antes de animar.
- **Erro: problemas co uso de `query`:**
 - Causa: non engadir `{ optional: true }` en elementos que poden non estar no DOM.
 - Solución: inclúe `{ optional: true }` ao usar `query`.
- **Erro: desempeño baixo en dispositivos antigos:**
 - Problema: animacións con moitas transicións complexas.
 - Solución: simplifica os estilos e reduce o número de elementos animados simultaneamente.

Rutas

Como xa sabemos, as aplicacións en Angular son páxinas únicas que actúan coma conxuntos de páxinas enrutadas no propio navegador, é dicir, son *Single Page Applications* (SPA). Estas aplicacións liberan ao servidor dunha boa parte do traballo, reducindo a cantidade de chamadas, mellorando a percepción de velocidade por parte do usuario e, con ela, a interactividade.

Nos proxectos de Angular 18, por defecto créase no cartafol raíz do proxecto un ficheiro TypeScript de nome `app.routes.ts` que fai uso de `RouterModule` (o módulo que contén toda a lóxica precisa para enrutar no navegador, así como para permitir a definición de rutas no array `Routes[]`). O contido do devandito TypeScript en Angular 18 é o seguinte:

```
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

O array de rutas recibe obxectos ruta con diversas propiedades de configuración, entre outras:

- **path:** trátase da dirección que resolve.
- **component:** é opcional e indica a compoñente que debe amosarse cando esa ruta sexa activada. O enrutado funciona establecendo un camiño a unha compoñente asociada. Seguidamente, a táboa de enrutado procésase de arriba cara abaixo e cando un camiño coincide coa ruta actual, párase e cárgase a compoñente.
- **redirectTo:** é outro parámetro opcional no que se permite facer redireccións duns enderezos a outros.

Rutas simples

Procederemos a crear unha serie de compoñentes para testar o funcionamento das rutas en Angular 17. Co gaio diso, situámonos na terminal no directorio `src/app/components` do noso proxecto e executamos os seguintes comandos:

```
ng g c portada
ng g c notFound
ng g c probas
```

O noso obxectivo é ter unha páxina enrutada de portada, outra de probas e que calquera outra ruta sexa redirixida cara á compoñente `NotFoundComponent`:

- <http://localhost:4200/>: accederá á compoñente `PortadaComponent`.
- <http://localhost:4200/portada/>: accederá á compoñente `PortadaComponent`.
- <http://localhost:4200/probas/>: accederá á compoñente `ProbasComponent`.

- <http://localhost:4200/xxx/>: accederá á compoñente `NotFoundComponent`.

Definindo as rutas

No arquivo TypeScript que comentamos anteriormente (**`app.routes.ts`**) introducimos as rutas indicadas xunta coa ligazón ás correspondentes compoñentes en función da ruta. Ademais facemos as importacións que sexan necesarias:

```
import { Routes } from '@angular/router';
import { PortadaComponent } from './portada/portada.component';
import { ProbasComponent } from './probas/probas.component';
import { NotFoundComponent } from './not-found/not-found.component';

export const routes: Routes = [
  { path: 'portada', title: 'Portada', component: PortadaComponent },
  { path: 'probas', title: 'Probas', component: ProbasComponent },
  { path: '', redirectTo: '/portada', pathMatch: 'full' },
  { path: '**', component: NotFoundComponent }, // Comodín para a páxina de erro
  404
];
```

Compoñente raíz

Procedemos agora a facer os cambios precisos no contido de cada unha das compoñentes para despois comentalos todos de xeito conxunto. Se lembramos, o arrinque da nosa aplicación en Angular faise na vista `app.component.html`. O código deste ficheiro deixámolo como se mostra a continuación:

```
<button routerLink="/" routerLinkActive="active"
ariaCurrentWhenActive="page">Portada</button> &nbsp;
<button routerLink="/probas" routerLinkActive="active"
ariaCurrentWhenActive="page">Probas</button>

<router-outlet></router-outlet>
```

Nese ficheiro podemos colocar un menú que, no noso caso e por simplicidade, vai consistir simplemente en dous botóns que acceden ás páxinas das compoñentes `portada` e a `probas` respectivamente.

Finalmente aparece unha nova etiqueta de `<router-outlet>`, que corresponde á directiva empregada para inserir a compoñente que corresponda en base ás rutas seleccionadas. Como sabemos, a idea xeral dunha SPA é ter unha única páxina que carga dinamicamente outras vistas. Normalmente esta páxina acostuma ter un menú de navegación, un pé de páxina e outras áreas comúns, e deixa un espazo para a carga dinámica. Para iso precisamos saber a compoñente a cargar e onde vai ser amosada, esa é a labor de *router outlet*.

Tamén podemos observar nos botóns a forma empregada en Angular para indicar a ruta enlazada nas correspondentes ligazóns (neste caso os botóns). Esa forma vai estar representada polo atributo `routerLink`.

No ficheiro **app.component.ts** importamos as clases `RouterLink` e `RouterLinkActive`, xa que son precisas para o correcto funcionamento dos enlaces ás rutas definidas anteriormente:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'rutas';
}
```

Compoñente «portada»

Nesta compoñente unicamente modificaremos o ficheiro **app.portada.html**, co seguinte contido:

```
<h1>Portada</h1>
<h2>Contido da Portada</h2>
```

Compoñente «not-found»

Pola súa parte o contido da compoñente para as rutas non buscadas **not-found.component.html** vai consistir nunha mensaxe de texto e un botón para voltar a **portada**.

```
<h2>Erro 404: sentímolo, pero a páxina solicitada non existe.</h2>
<a routerLink="/">
  Voltar á Portada
</a>
```

E no ficheiro **not-found.component.ts** importaremos así mesmo a clase `RouterLink`:

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-not-found',
  standalone: true,
  imports: [RouterLink],
  templateUrl: './not-found.component.html',
  styleUrls: ['./not-found.component.css']
})

export class NotFoundComponent {}
```

Compoñente «probas»

No caso do ficheiro `probas.component.html` colocamos o contido que queiramos, por exemplo:

```
<h1>Sección de probas</h1>
<h3>Contido da sección de probas</h3>
```

Resultado

O único que nos queda xa é probar que todo funciona como esperabamos. Velaquí un resultado aproximado dende cada unha das vistas propostas:

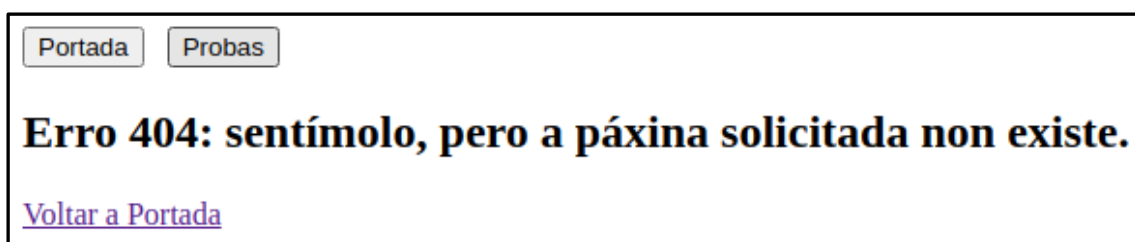
- Accedendo dende <http://localhost:4200/home> ou, simplemente, <http://localhost:4200/>.



- Accedendo dende <http://localhost:4200/probas>.



- Accedendo dende calquera outro enderezo descendente do noso servidor.



Exemplo T06E11

Neste exemplo introdúcese a navegación básica con Angular Router: defínense rutas para dúas vistas (*portada* e *probos*), establecendo tamén unha redirección dende a ruta baleira á *portada* e configurando unha ruta comodín para xestionar camiños non existentes; a navegación faise mediante enlaces declarativos e a vista activa móstrase na saída do enrutador.

O exemplo ten estes obxectivos didácticos:

- Comprender como declarar rutas básicas, redirección inicial e ruta comodín.
- Entender o papel da saída do enrutador como contedor da vista activa.
- Usar enlaces declarativos para cambiar de ruta sen recargar a páxina.

Rutas aniñadas

Angular permítenos tamén aniñar rutas, de xeito que a nosa SPA emule o comportamento dun sitio Web composto por múltiples páxinas independentes con cadansúa ruta.

Co gallo de acadar isto, partimos do exemplo anterior e procedemos a crear dúas compoñentes fillas da compoñente *proba*:

```
ng g c probas/probas-a
ng g c probas/probas-b
```

Nos seus ficheiros HTML incorporamos un texto sinxelo, simplemente para comprobar o correcto funcionamento da aplicación. Por exemplo, o ficheiro **probos-a.component.html**:

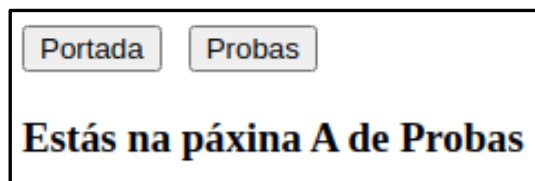
```
<h3>Estás na páxina A de Probos</h3>
```

Finalmente, modificamos o ficheiro de rutas (**app.routes.ts**) do xeito que segue:

```
import { Routes } from '@angular/router';
import { PortadaComponent } from '../portada/portada.component';
import { ProbosComponent } from '../probos/probos.component';
import { NotFoundComponent } from '../not-found/not-found.component';
import { ProbosAComponent } from '../probos/probos-a/probos-a.component';
import { ProbosBComponent } from '../probos/probos-b/probos-b.component';

export const routes: Routes = [
  { path: 'portada', title: 'Portada', component: PortadaComponent },
  { path: 'probos', children: [
    { path: 'probos-a', title: 'Probos - Probos A', component: ProbosAComponent },
    { path: 'probos-b', title: 'Probos - Probos B', component: ProbosBComponent }
  ] },
  { path: '', redirectTo: '/portada', pathMatch: 'full' },
  { path: '**', component: NotFoundComponent }, // Comodín para a páxina de erro
  404
];
```

Neste caso, se accedemos por exemplo á ruta <http://localhost:4200/probas/probas-a> veremos algo así:



Exemplo T06E12

Neste exemplo defínese unha ruta pai con dúas rutas fillas baixo o mesmo camiño, renderizando a compoñente pai e un `router-outlet` interno onde se cargan as fillas ao navegar a `/probas/probas-a` ou `/probas/probas-b`; a configuración correcta agrupa `component` e `children` nun único obxecto da ruta pai.

Este exemplo persegue os seguintes obxectivos didácticos:

- Comprender a relación pai-filla entre rutas e o uso dun `router-outlet` anidado.
- Aprender a estruturar a configuración con `children` para agrupar subrutas baixo un camiño común.
- Diferenciar a navegación absoluta e relativa ao enlazar dende a ruta pai ás fillas.

Paso de parámetros

Neste apartado veremos como pasar un parámetro a través dunha ruta, de xeito que a compoñente situada no outro lado da mesma poida recuperalo e procesalo. É, polo tanto, máis un xeito de intercambiar información entre compoñentes.

Angular permítenos pasar parámetros a rutas de dous xeitos:

- **Rutas parametrizadas** na súa definición.
- **Rutas non parametrizadas** na súa definición.

Rutas parametrizadas

Co gallo de explicar o procedemento a seguir, partiremos do primeiro exemplo que realizamos neste capítulo (o correspondente ás rutas simples) e modificaremos os seguintes ficheiros:

- **`app.routes.ts`**: modificamos a liña correspondente á ruta conducente á compoñente `probas`, de xeito que reciba un parámetro de nome `id`.

```
import { Routes } from '@angular/router';
import { PortadaComponent } from '../portada/portada.component';
import { ProbasComponent } from '../probas/probas.component';
import { NotFoundComponent } from '../not-found/not-found.component';
```

```
export const routes: Routes = [
  { path: 'portada', title: 'Portada', component: PortadaComponent },
  { path: 'probas/:id', title: 'Probas', component: ProbasComponent },
  { path: '', redirectTo: '/portada', pathMatch: 'full' },
  { path: '**', component: NotFoundComponent }, // Comodín para a páxina de erro
];
```

- **app.component.html:** modificamos a liña correspondente ao botón que conduce á ruta da compoñente probas.

```
<button routerLink="/" routerLinkActive="active"
ariaCurrentWhenActive="page">Portada</button> &nbsp;
<button [routerLink]="['/probas',5]" routerLinkActive="active"
ariaCurrentWhenActive="page">Probas</button>

<router-outlet></router-outlet>
```

- **probas.component.ts:** engadimos unha propiedade `id` e un comportamento `OnInit` de xeito que se recupere o valor do parámetro recibido e se asigne sobre a propiedade.

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-probas',
  standalone: true,
  imports: [],
  templateUrl: './probas.component.html',
  styleUrls: ['./probas.component.css']
})
export class ProbasComponent implements OnInit {
  id: number;

  constructor(private ruta: ActivatedRoute) {}

  ngOnInit(): void {
    this.ruta.params.subscribe(parametros => {
      this.id = parseInt(parametros['id']); // Convertimos o parámetro 'id'
      recibido na ruta nun número.
    })
  }
}
```

- **probas.component.html:** amosamos a propiedade `id` para asegurar que a ruta funcionou correctamente.

```
<h1>Sección de probas</h1>
<h3>Contido da sección de probas</h3>
<p>O valor do id recibido foi: {{ id }}</p>
```

Agora se quixeramos pasar un parámetro á nosa compoñente probas poderíamos poñer (supoñamos que estamos a traballar sobre o servidor de desenvolvemento de Angular):

```
http://localhost:4200/probas/17
```

Exemplo T06E13

Neste exemplo amplíase a configuración de rutas con `children` para aceptar parámetros dinámicos no camiño (p.ex., `/probas/probas-a/:cousa` e `/probas/probas-b/:numero1/:numero2`) e recupéranse nas compoñentes fillas a través de `ActivatedRoute` e a subscrición a `params`.

Este exemplo persegue os seguintes obxectivos didácticos:

- Comprender como definir segmentos con parámetros nunha ruta filla e como navegalos.
- Ler parámetros de ruta mediante `ActivatedRoute.params` e usalos na vista ou na lóxica.
- Diferenciar entre valores de parámetros como cadeas e a necesidade de convertelos cando se requiren números.

Rutas con parámetros de consulta

Para operar deste xeito empregamos os **QueryParams** (parámetros de consulta), que son valores que se engaden á URL dunha aplicación empregando a nomenclatura clásica das aplicacións Web.

Co gallo de explicar o procedemento a seguir, partiremos dunha variante do primeiro exemplo (empregado para explicar as rutas simples):

- **`app.routes.ts`**: non é preciso modificar nada.
- **`app.component.html`**: aquí podemos indicar algúns parámetros a pasar, mediante o *binding* do atributo `queryParams`:

```
<!-- Este botón redirixe á páxina de portada -->
<button routerLink="/" routerLinkActive="active">Portada</button>

<!-- Este botón redirixe á páxina de probas pasándolle coma parámetro un obxecto
que ten dúas propiedades ("numero1" e "numero2") -->
<button [routerLink]="['/probas']" [queryParams]="{ numero1: 15, numero2: 20 }"
routerLinkActive="active">Probas</button>

<!-- Este botón redirixe á páxina de probas pasándolle parámetros a través de TS -
->
<button (click)="navegar()">Outro camiño a Probas</button>

<!-- Dentro de 'router-outlet' cargaránse as distintas compoñentes cando a súa
ruta sexa accedida -->
<router-outlet></router-outlet>
```

- **`probas.component.ts`**: engadimos un comportamento `OnInit` de xeito que se recupere o valor dos parámetros recibidos e se asigne sobre certas propiedades de clase.

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
```

```

@Component({
  selector: 'app-probas',
  standalone: true,
  imports: [],
  templateUrl: './probas.component.html',
  styleUrls: ['./probas.component.css']
})
export class ProbasComponent implements OnInit {
  numero1: number = 0;
  numero2: number = 0;

  constructor(private ruta: ActivatedRoute) {}

  ngOnInit(): void {
    // Capturamos os parámetros recibidos a través da ruta mediante unha
    // subscripción ao Observable "queryParams"
    this.ruta.queryParams.subscribe((parametros) => {
      this.numero1 = parametros["numero1"];
      this.numero2 = parametros["numero2"];
    });
  }
}

```

- **probas.component.html:** amosamos a propiedade `id` para asegurar que a ruta funcionou correctamente.

```

<h1>Sección de probas</h1>
<p>Contido da sección de probas</p>
<p>O resultado da multiplicación é {{ numero1 * numero2 }}</p>

```

Agora se quixeramos pasar parámetros á nosa compoñente `probas` poderíamos empregar a mesma dinámica que cando traballamos con calquera linguaxe Web de servidor típica (supoñamos que estamos a traballar sobre o servidor de desenvolvemento de Angular):

```
http://localhost:4200/probas?numero1=15&numero2=20
```

Exemplo T06E14

Neste exemplo móstrase como ler parámetros de consulta dunha URL usando `ActivatedRoute.queryParams`. A compoñente recolle `numero1` e `numero2` da cadea de consulta e emprega os seus valores na vista; cómpre lembrar que chegan como cadeas e poden precisar conversión de tipo segundo o uso do que sexan obxecto.

Os obxectivos didácticos deste exemplo son:

- Comprender a diferenza entre parámetros de ruta e parámetros de consulta.
- Empregar `ActivatedRoute.queryParams` para subscribirse e obter cambios en tempo real.
- Recoñecer que os valores veñen como cadeas e converter a tipo numérico cando sexa preciso.
- Aplicar os parámetros de consulta para realizar operacións e actualizar a vista.

Rutas con *guards*

Os *guards* en Angular son un tipo especial de servizo (levan o decorador `@Injectable`) utilizado para controlar a navegación dentro dunha aplicación, permitindo ou denegando o acceso ás rutas en base a condicións específica como, por exemplo, se un usuario está autenticado.

Hai diversos tipos de *guards*, segundo a funcionalidade ofrecida:

- **CanActivate:** comproba se un usuario pode visitar a ruta á cal se aplica.
- **CanActivateChild:** comproba se un usuario pode visitar as rutas fillas daquela á cal se aplica.
- **CanDeactivate:** comproba se un usuario pode abandonar a ruta actual.
- **CanLoad:** determina se un módulo pode ser cargado de xeito preguiceiro⁴.

Os *guards* créanse mediante o comando:

```
ng generate guard nome_guard
```

Ou en forma reducida:

```
ng g g nome_guard
```

⁴ No orixinal: *lazy-loading*.

Vexámolo cun exemplo, para o que crearemos unha aplicación *standalone* coas seguintes compoñentes:

- **login:** accesible a través da ruta por defecto `/`.
- **portada:** accesible a través da ruta `/portada`.
- **restrinxido:** accesible a través da ruta `/restrinxido`.

O obxecto da nosa aplicación é ter unha simulación de *login* de xeito que deamos a opción aos usuarios de ingresar un nome de usuario e contrasinal ou entrar coma convidados. Unha vez accedido pasarase a cargar a compoñente **portada**, dende a que verán unha ligazón á compoñente **restrinxido**. No caso de que entren como convidados, na portada impediráselles o acceso á compoñente **restrinxido**.

A compoñente login

Esta compoñente disporá dun formulario de ingreso típico (cun campo para introducir o usuario e outro para introducir o contrasinal) e dous botóns: un para iniciar sesión cos datos do formulario e outro para inicio de sesión anónimo. No caso de iniciar sesión cun usuario, gardarase o mesmo en `sessionStorage` con clave `usuario`.

Código HTML

```
<div class="contedor-login">
  <form class="formulario-login" [formGroup]="formularioLogin">
    <div class="formulario-grupo">
      <label for="usuario">Usuario:</label>
      <input type="text" id="usuario" placeholder="Nome de usuario"
formControlName="login">
    </div>
    <div class="form-group">
      <label for="contrasinal">Contrasinal:</label>
      <input type="password" id="contrasinal" placeholder="Contrasinal"
formControlName="contrasinal">
    </div>
    <button type="button" id="boton-login" (click)="iniciarSesion()">Iniciar
sesión</button>
    <button type="button" id="boton-convidado"
(click)="entrarConvidado()">Entrar como convidado</button>
  </form>
</div>
```

Código CSS

```
.contedor-login {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
```

```

.formulario-login {
  width: 300px;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 5px;
  background-color: #f9f9f9;
}

.formulario-grupo {
  margin-bottom: 15px;
}

label {
  display: block;
  margin-bottom: 5px;
}

input {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

button {
  width: 100%;
  padding: 10px;
  border: none;
  border-radius: 5px;
  margin-top: 10px;
  cursor: pointer;
}

#boton-login {
  background-color: #007bff;
  color: white;
}

#boton-convidado {
  background-color: #6c757d;
  color: white;
}

```

Código TS

```

import { Component } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule } from '@angular/forms';
import { Router } from '@angular/router';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})

export class LoginComponent {
  formularioLogin: FormGroup;

  constructor(private fb: FormBuilder, private router: Router) {
    this.formularioLogin = fb.group({
      login: [],
      contrasinal: []
    })
  }
}

```

```

    iniciarSesion(): void {
        sessionStorage.setItem('usuario', this.formularioLogin.value.login); // Simula
o inicio de sesión gardando o usuario en sessionStorage
        this.router.navigate(['/portada']); // Redirixir á páxina de portada
    }

    entrarConvidado(): void {
        // Procede como convidado sen gardar nada en sessionStorage
        this.router.navigate(['/portada']); // Redirixir á páxina de portada
    }
}

```

A compoñente portada

Esta compoñente amosa unha ligazón á compoñente restrinxido e un botón que permite pechar a sesión, eliminando a clave `usuario` de `sessionStorage` e retornando á páxina de *login*.

Código HTML

```

<div class="contedor-portada">
    <h1>Iniciou vostede sesión como {{ usuario ? ' ' + usuario : ' convidado'
}}</h1>
    <a [routerLink]="['/restrinxido']">Ver contido restrinxido</a>
    <a (click)="pecharSesion()">Pegar Sesión</a>
</div>

```

Código CSS

```

.contedor-portada {
    text-align: center;
    margin-top: 50px;
}

a {
    display: inline-block;
    margin: 20px;
    padding: 10px;
    background-color: #007bff;
    color: white;
    text-decoration: none;
    border-radius: 5px;
}

```

Código TS

```

import { Component, OnInit } from '@angular/core';
import { Router, RouterLink } from '@angular/router';

@Component({
    selector: 'app-portada',
    standalone: true,
    imports: [RouterLink],
    templateUrl: './portada.component.html',
    styleUrls: ['./portada.component.css']
})

```

```

export class PortadaComponent implements OnInit {
  usuario: string | null = null; // A propiedade usuario pode ter un valor string
  (se houbo inicio de sesión) ou null (se se entrou coma convidado)

  constructor(private router: Router) { }

  pecharSesion(): void {
    sessionStorage.removeItem('usuario');
    this.router.navigate(['/']); // Redirixir á páxina de login
  }

  ngOnInit(): void {
    this.usuario = sessionStorage.getItem('usuario');
  }
}

```

A compoñente restrinxido

Esta compoñente unicamente amosa un texto, para comprobar que efectivamente foi accedida, polo que non é preciso engadir aquí o seu código.

O ficheiro `app.routes.ts`

Como sabemos, neste ficheiro defínense as rutas que dirixen ás distintas compoñentes da nosa aplicación:

```

import { Routes } from '@angular/router';
import { LoginComponent } from '../login/login.component';
import { PortadaComponent } from '../portada/portada.component';
import { RestrinxidoComponent } from '../restrinxido/restrinxido.component';
import { RestrinxidoGuard } from '../guards/restrinxido-guard.guard';

export const routes: Routes = [
  { path: '', title: 'Inicio de sesión', component: LoginComponent },
  { path: 'portada', title: 'Portada', component: PortadaComponent },
  { path: 'restrinxido', title: 'Contido Restrinxido', component:
    RestrinxidoComponent, canActivate: [RestrinxidoGuard] } // Incorporamos o noso
    guard á ruta para controlar o acceso á mesma
];

```

O guard `restrinxido-guard`

O noso *guard* fai uso do método `canActivate` para determinar, en base á clave `usuario` de `sessionStorage`, se pode accederse ou non á ruta, redirixindo á páxina de *login* en caso negativo.

```

import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

export const RestrinxidoGuard: CanActivateFn = (route, state) => {
  let router = inject(Router); // Inxectamos o Router (singleton) para poder
  redirixir o tráfico
  let usuario = sessionStorage.getItem("usuario"); // Recuperamos o valor do
  usuario actual dende sessionStorage
}

```

```

// Comprobamos se había usuario gardado en sessionStorage
if (usuario) {
  return true;
} else {
  router.navigate(['']); // Se non hai usuario logueado impedimos o acceso á
  ruta e rediriximos á páxina de login
  return false;
}
};

```

Exemplo T06E15

Neste exemplo ilústrase como protexer rutas con un *guard* de tipo `canActivate` definido como función. O *guard* comproba se existe un usuario en `sessionStorage`; se existe permite o acceso á ruta restrinxida, e se non existe bloquea a navegación e redirixe á páxina de inicio de sesión. A páxina de *login* establece o valor de usuario en `sessionStorage` tras enviar o formulario; a portada permite pechar sesión eliminando ese valor. A ruta *restrinxido* queda así protexida polo *guard*.

O exemplo ten estes obxectivos didácticos:

- Comprender o fluxo de navegación con `canActivate` (permitir/denegar acceso segundo estado).
- Implementar un *guard* funcional empregando `inject` para obter `Router`.
- Aplicar redirección programática cando o acceso é denegado.
- Usar `sessionStorage` para manter un estado de sesión sinxelo e eliminalo en peche de sesión.
- Integrar o *guard* co enrutado básico e un formulario reactivo de inicio de sesión.

Carga preguiceira de rutas

A carga preguiceira permite mellorar o rendemento cargando só os módulos necesarios para unha determinada ruta:

```

const routes: Routes = [
  { path: 'usuarios', loadChildren: () =>
    import('./usuarios/usuarios.module').then(m => m.UsuariosModule) }
];

```

Erros e problemas habituais

Neste apartado veremos algúns dos erros máis habituais que poden xurdir no uso de rutas en Angular:

- **Erro: `"cannot find module '...'"`.**
 - Causa: ruta incorrecta en `loadChildren`.
 - Solución: verifica a ruta ao módulo.

- **Erro: "*cannot match any routes*".**
 - Causa: ruta non definida no array `Routes`.
 - Solución: asegúrate de cubrir todas as rutas posibles.
- **Erro: "*outlet is not a known element*".**
 - Causa: falta importar `RouterModule` no módulo correspondente.
 - Solución: engade `RouterModule` ao array `imports`.
- **Erro: problemas con *guards*.**
 - Causa: configuración incorrecta ou retorno dun valor non booleano.
 - Solución: verifica que o *guard* devolva `true` ou `false`.
- **Erro: bucle de redirección.**
 - Causa: configuración de redireccións mal definida.
 - Solución: asegúrate de que as rutas de redirección non se enlacen en bucle.

Formularios reactivos

Nas primeiras versións de Angular o tratamento de formularios facíase principalmente a través de enlazamento bidireccional. Ese método, que facilita enormemente o desenvolvemento de formularios fronte as técnicas tradicionais empregadas en JavaScript tiña, porén, un grande custe en termos de escalabilidade, xa que implicaba un maior tempo en execución ao tempo que dificultaba a validación e o mantemento de formularios complexos.

A solución ao problema anterior foi introducida en Angular 2, e consistiu en desacoplar o modelo e a vista, introducindo unha nova capa que pasaría ser a encargada da xestión esa dobre ligazón de datos. Aos formularios que empregan esa técnica coñéceselles coma **formularios reactivos**, e o seu comportamento ven determinado polos servizos e directivas do módulo `ReactiveFormsModule`, incluído na biblioteca `@angular/forms`.

O anterior permite:

- Un control máis preciso sobre o estado e a validación dos formularios.
- A capacidade de definir validacións complexas e personalizadas.
- Maior rendemento en formularios extensos.

Creación de formularios reactivos

Para probar un exemplo e entender mellor os formularios reactivos, partiremos do exemplo que creamos no punto anterior para as rutas simples, e crearemos un cartafol de nome `Seguridade` dentro de `src/app` no que engadiremos un nova compoñente chamada `rexistro` que empregaremos, coma o seu nome indica, para rexistrar aos usuarios.

Ademais do anterior, engadiremos a correspondente ruta de rexistro no arquivo `app.routes.ts`:

```
import { Routes } from '@angular/router';
import { PortadaComponent } from './portada/portada.component';
import { ProbasComponent } from './probas/probas.component';
import { NotFoundComponent } from './not-found/not-found.component';
import { RexistroComponent } from './seguridade/rexistro/rexistro.component';

export const routes: Routes = [
  { path: 'portada', title: 'Portada', component: PortadaComponent },
  { path: 'probas', title: 'Probas', component: ProbasComponent },
  { path: 'rexistro', title: 'Rexistro', component: RexistroComponent },
  { path: '', redirectTo: '/portada', pathMatch: 'full' },
  { path: '**', component: NotFoundComponent }, // Comodín para a páxina de erro
  404
];
```

Engadiremos tamén un novo botón no menú para poder rexistrarse. Velaquí os cambios a incluír en ficheiro `app.component.html`:

```
<button routerLink="/" routerLinkActive="active"
ariaCurrentWhenActive="page">Portada</button> &nbsp;&nbsp;&nbsp;
<button routerLink="/probas" routerLinkActive="active"
ariaCurrentWhenActive="page">Probas</button> &nbsp;&nbsp;&nbsp;
<button routerLink="/rexistro" routerLinkActive="active"
ariaCurrentWhenActive="page">Rexistro</button>

<router-outlet></router-outlet>
```

Neste punto precisamos do servizo `FormBuilder`, que vai encargarse de desacoplar o modelo da vista para aquelas compoñentes que así o precisen. Permitiranos crear un formulario mediante un grupo de controis `FormGroup` ao tempo que realizará un seguimento do valor e estado de cambio así como da validez dos datos. Para poder empregar o devandito servizo precisaremos importar o módulo de Angular no que ven declarado (`ReactiveFormsModule`).

Procedemos pois ao ficheiro `rexistro.component.ts`, no que incorporaremos o módulo mencionado. Así, definimos unha variable de nome `formGroup` que conterá o formulario co grupo de controis de xeito que cada control posuirá nome e unha configuración de seu. No caso de querer establecer valores iniciais a cada un dos controis poderémolo facer dende aquí:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-rexistro',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './rexistro.component.html',
  styleUrls: ['./rexistro.component.css']
})
export class RexistroComponent implements OnInit {
  grupoFormulario: FormGroup;

  constructor(private construtorFormularios: FormBuilder) { }

  ngOnInit(): void {
    this.construirFormulario();
  }

  construirFormulario(): void {
    var lonxitudeData = 10;
    var hoxe = new Date().toISOString().substring(0, lonxitudeData);
    var nome = 'Xan das Bolas';

    this.grupoFormulario = this.construtorFormularios.group({
      rexistrado: hoxe,
      nome: nome.toLowerCase(),
      email: 'xandasbolas@meu-correo.gal',
      contrasinal: 'abc123..'
    })
  }
}
```

```

public rexistrar() {
  var usuario = this.grupoFormulario.value;
  console.log(usuario);
}
}

```

Todo este traballo previo no arquivo TypeScript permite unha vista moito máis limpa. O único que precisamos sinalar no formulario no HTML é a ligazón ao formulario. Co gaio disto empregamos dúas directivas que veñen dentro do módulo reactivo:

- **[formGroup]="obxectoFormulario"**: para enlazar o formulario no seu conxunto.
- **formControlName="nomeDoControl"**: para enlazar cada control.

Resultando as modificacións na arquivo HTML `registro.component.html` da maneira que segue:

```

<br><br>
<form [formGroup]="grupoFormulario">
  <label for="rexistrado">Rexistrado na data:</label>
  <input name="rexistrado" formControlName="rexistrado" type="date" />
  <br><br>
  <label for="nome">Nome: </label>
  <input name="nome" formControlName="nome" type="text" />
  <br><br>
  <label for="email">E-mail: </label>
  <input name="email" formControlName="email" type="email" />
  <br><br>
  <label for="contrasinal">Contrasinal: </label>
  <input name="contrasinal" formControlName="contrasinal" type="password" />
  <br><br>
  <button (click)="rexistrar()">
    Rexistrar
  </button>
</form>

```

Podemos xa visualizar a nosa aplicación a través do enderezo <http://localhost:4200/registro/>:

Exemplo T06E16

Neste exemplo créase un formulario reactivo empregando `FormBuilder` e `FormGroup`, asociándoo á vista con `[formGroup]` e `formControlName`. Obtéñense os cambios do formulario a través de `valueChanges`, aplicando `debounceTime` para evitar emisións excesivas, e lógrase un envío controlado mediante `(ngSubmit)` evitando o comportamento por defecto do navegador.

O exemplo ten estes obxectivos didácticos:

- Comprender como definir un `FormGroup` e inicializar controis con `FormBuilder`.
- Vincular controis do formulario á vista mediante `formControlName` e `[formGroup]`.
- Empregar `valueChanges` con `debounceTime` para reaccionar a modificacións do formulario.
- Xestionar o envío con `(ngSubmit)` e `preventDefault` para tratar os datos sen recargar a páxina.

Validación de formularios reactivos

Unha vez teñamos o noso formulario procederemos a realizar a validación de datos, xa que constitúe o primeiro control aos erros introducidos polos usuarios xa sexan estes involuntarios ou á mantenta. Un exemplo claro sería a emrega de novos atributos como por exemplo, o uso de `required` para indicar que esa entrada era obrigatoria. Nos formularios reactivos esa validación trasládase á configuración de cada control onde estableceremos unha ou varias regras. Unha forma de establecer validacións é empregando baterías de regras que veñen predefinidas como funcións no obxecto `Validators` da librería `@angular/forms` do framework.

É preciso facer a importación oportuna no arquivo `registro.components.ts` e modificar a función de control do formulario:

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from
'@angular/forms';

@Component({
  selector: 'app-registro',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './registro.component.html',
  styleUrls: ['./registro.component.css']
})

export class RegistroComponent implements OnInit {
  grupoFormulario: FormGroup;

  constructor(private construtorFormularios: FormBuilder) { }

  ngOnInit(): void {
    this.construirFormulario();
  }
}
```

```

construirFormulario(): void {
    var lonxitudeData = 10;
    var hoxe = new Date().toISOString().substring(0, lonxitudeData);
    var nome = 'Xan das Bolas';
    var lonxMinContrasinal = 4;

    this.grupoFormulario = this.construtorFormularios.group({
        rexistrado: hoxe,
        nome: [nome.toLowerCase(), Validators.required], // O campo pasa ser
obrigatorio
        email: ['xandasbolas@meu-correo.gal', [Validators.required,
Validators.email ]], // O campo pasa ser obrigatorio e con validación de tipo e-
mail
        contrasinal: ['abc123..', [Validators.required,
Validators.minLength(lonxMinContrasinal)]] // O campo pasa ser obrigatorio e con
validación de lonxitude mínima
    });
}

public rexistrar() {
    var usuario = this.grupoFormulario.value;
    console.log(usuario);
}
}

```

Unha vez establecidas as regras, é preciso aplicalas e avisar ao usuario no caso que sexan incumpridas. É importante lembrar que os formularios e controis interactivos están xestionados por máquinas de estados que determinan a situación de cada control e do formulario, polo que debemos programar en consonancia a tal estrutura. O sistema de validación para xestión do estado de cada control e do formulario presenta os seguintes estados:

- **valid:** o control pasou todas as validacións.
- **invalid:** o control fallou polo menos nunha regra.
- **pending:** o control está en medio dun proceso de validación.
- **disabled:** o control está desactivado e exento de validación.

Cando o control incumpre algunha regra de validación, refléxase esta situación na súa propiedade `errors` que é un obxecto cunha propiedade por cada regra insatisfeita e un valor ou mensaxe de axuda.

Ademais da anterior validación, os controis e o formulario sométense a outra máquina de estados para monitorizar o valor do control e os seus cambios, podendo estar entre un dos seguintes estados:

- **pristine:** o valor do control non foi modificado polo usuario.
- **dirty:** o usuario modificou o valor do control.
- **touched:** o usuario tocou o control lanzando un evento `blur` ao saír.
- **untouched:** o usuario non tocou e saíu do control sen lanzar ningún evento `blur`.

Chegados a este punto é esencial comprobar que, se a validación dalgún control falla, é preciso inhabilitar o botón de envío. A modificación de `registro.component.html` será como sigue:

```

<br><br>
<form [formGroup]="grupoFormulario">
  <label for="rexistrado">Rexistrado na data:</label>

```

```

<input name="rexistrado" formControlName="rexistrado" type="date" />
<br><br>
<label for="nome">Nome: </label>
<input name="nome" formControlName="nome" type="text" />
<br><br>
<label for="email">E-mail: </label>
<input name="email" formControlName="email" type="email" />
<br><br>
<label for="contrasinal">Contrasinal: </label>
<input name="contrasinal" formControlName="contrasinal" type="password" />
<br><br>
<button (click)="rexistrar()" [disabled]="grupoFormulario.invalid">
  Rexistrar
</button>
</form>

```

O sistema de xestión dos controis do formulario oculta a parte máis valiosa, é dicir, os datos do usuario na propiedade `value` do formulario. Trátase dun obxecto coas mesmas propiedades usadas durante a definición do formulario e contendo o valor actual de cada control asociado.

Exemplo T06E17

Este exemplo amplía o formulario reactivo anterior engadindo validacións con `Validators` (`required`, `maxlength`, `email`), valores iniciais e mensaxería de erro condicionada ao estado `touched/invalid` de cada control. Emprega *getters* para simplificar o acceso aos controis no *template*, marca todos os campos como tocados cando o formulario é inválido ao enviar e inclúe un contador de caracteres con realce visual cando se supera o límite.

Os obxectivos didácticos que se pretende acadar con este exemplo son:

- Comprender como engadir validacións síncronas a controis dun `FormGroup` con `Validators`.
- Amosar mensaxes de erro no *template* usando `*ngIf`, `hasError` e a propiedade `touched`.
- Inicializar controis con valores por defecto (p.ex. a data de hoxe).
- Empregar *getters* na compoñente para acceder aos controis dende a vista con claridade.
- Xestionar o envío validando formulario e usando `markAllAsTouched` cando é inválido.

Exemplo T06E18

Este exemplo adapta un formulario reactivo engadindo estilos e patróns de **Bulma CSS**: usa as clases `input`, `field`, `control`, `icon` e modificadores `is-success/is-danger` ligados ao estado dos `FormControl` a través de *getters*. Mantén as validacións (`required`, `maxlength`, `email`), engade validación específica para o `select` de gravidade cun valor sentinela e mostra iconas e mensaxes de erro condicionais segundo `touched/invalid`. O envío comproba tanto a validez global do formulario como a de gravidade e marca todos os campos como tocados cando é inválido.

O exemplo pretende acadar estes obxectivos didácticos:

- Integrar Bulma en formularios reactivos mantendo a lóxica de validación en Angular.
- Ligar clases CSS de estado (`is-success/is-danger`) ao estado dos controis con *getters*.
- Mostrar iconas e axuda contextual en función do estado `touched/invalid` de cada campo.
- Aplicar unha validación *ad hoc* para *selects* mediante un valor sentinela e un *getter* específico.
- Xestionar o envío validando, imprimindo os datos e marcando campos como tocados cando corresponda.

Validacións personalizadas

Ás validacións integradas que vimos no punto anterior pódense engadir outras creadas polo programador. Por exemplo, se queremos introducir un novo control de validación no contrasinal de xeito que este teña que comezar por un número, engadiremos un método que recibirá o control a validar, tendo en conta:

- O resultado será `null` no caso de que a validación sexa correcta.
- O resultado será un fallo en calquera outro caso.

Seguidamente, introducimos os cambios correspondentes nas funcións correspondentes do arquivo `rexistro.components.ts`:

```
import { Component, OnInit } from '@angular/core';
import { AbstractControl, FormBuilder, FormGroup, ReactiveFormsModule, Validators
} from '@angular/forms';

@Component({
  selector: 'app-rexistro',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './rexistro.component.html',
  styleUrls: ['./rexistro.component.css']
})
export class RexistroComponent implements OnInit {
  grupoFormulario: FormGroup;

  constructor(private construtorFormularios: FormBuilder) { }

  ngOnInit(): void {
    this.construirFormulario();
  }

  construirFormulario(): void {
    var lonxitudeData = 10;
    var hoxe = new Date().toISOString().substring(0, lonxitudeData);
    var nome = 'Xan das Bolas';
    var lonxMinContrasinal = 4;
```

```

    this.grupoFormulario = this.construtorFormularios.group({
      rexistrado: hoxe,
      nome: [nome.toLowerCase(), Validators.required], // O campo pasa ser
obligatorio
      email: ['xandasbolas@meu-correo.gal', [ Validators.required,
Validators.email ]], // O campo pasa ser obligatorio e con validación de tipo e-
mail
      contrasinal: ['abc123..', [Validators.required,
Validators.minLength(lonxMinContrasinal), this.validarContrasinal]] // O campo
pasa ser obligatorio e con validación de lonxitude mínima
    });
  }

  private validarContrasinal(control: AbstractControl) {
    var contrasinal = control.value;
    var erro: any = null;

    // Comprobamos que o primeiro carácter é un número
    if (!parseInt(contrasinal[0])) {
      erro = { ...erro, number: 'Debe comezar cun número' };
    }

    return erro;
  }

  public rexistrar() {
    var usuario = this.grupoFormulario.value;
    console.log(usuario);
  }
}

```

Visualizar as validacións

A validación particular para cada control permite, como dixemos anteriormente, informar ao usuario do fallo concreto. Para iso é bo crear unha función auxiliar no noso TypeScript que amose os erros de validación, de xeito que finalmente recollamos eses erros na vista (no noso HTML) e sexan presentados ao usuario cos correspondentes `<span>`. Indicamos a continuación o arquivo TypeScript (`rexistro.component.ts`) completo co novo código que acabamos de comentar:

```

import { Component, OnInit } from '@angular/core';
import { AbstractControl, FormBuilder, FormGroup, ReactiveFormsModule, Validators
} from '@angular/forms';

@Component({
  selector: 'app-rexistro',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './rexistro.component.html',
  styleUrls: ['./rexistro.component.css']
})

export class RexistroComponent implements OnInit {
  grupoFormulario: FormGroup;

  constructor(private construtorFormularios: FormBuilder) { }

  ngOnInit(): void {
    this.construirFormulario();
  }
}

```

```

    construirFormulario(): void {
        var lonxitudeData = 10;
        var hoxe = new Date().toISOString().substring(0, lonxitudeData);
        var nome = 'Xan das Bolas';
        var lonxMinContrasinal = 4;

        this.grupoFormulario = this.construtorFormularios.group({
            rexistrado: hoxe,
            nome: [nome.toLowerCase(), Validators.required], // O campo pasa ser
obligatorio
            email: ['xandasbolas@meu-correo.gal', [Validators.required,
Validators.email ]], // O campo pasa ser obligatorio e con validación de tipo e-
mail
            contrasinal: ['abc123..', [Validators.required,
Validators.minLength(lonxMinContrasinal), this.validarContrasinal]] // O campo
pasa ser obligatorio e con validación de lonxitude mínima
        });
    }

    private validarContrasinal(control: AbstractControl) {
        var contrasinal = control.value;
        var erro: any = null;

        // Comprobamos que o primeiro carácter é un número
        if (!parseInt(contrasinal[0])) {
            erro = { ...erro, number: 'Debe comezar cun número' };
        }

        return erro;
    }

    public rexistrar() {
        var usuario = this.grupoFormulario.value;
        console.log(usuario);
    }

    public obterErro(nomecontrol: string): string {
        var erro = '';
        var control = this.grupoFormulario.get(nomecontrol);

        if(control.touched && control.errors != null) {
            erro = JSON.stringify(control.errors);
        }

        return erro;
    }
}

```

O código completo da nosa vista no arquivo `registro.component.html` será o seguinte:

```

<br><br>
<form [formGroup]="formGroup">
  <section>
    <label for="registeredOn">Registered On</label>
    <input name="registeredOn"
      formControlName="registeredOn"
      type="date" />
  </section>
  <section>
    <label for="name">Name</label>
    <input name="name"
      formControlName="name"
      type="text" />
    <span>{{ getError('name') }}</span>
  </section>

```

```

<section>
  <label for="email">E-mail</label>
  <input name="email"
        formControlName="email"
        type="email" />
  <span>{{ getError('email') }}</span>
</section>
<section>
  <label for="password">Password</label>
  <input name="password"
        formControlName="password"
        type="password" />
  <span>{{ getError('password') }}</span>
</section>
<section>
  <button (click)="register()"
        [disabled]="formGroup.invalid">
    Rexistrar
  </button>
</section>
</form>

```

O resultado pode verse na figura seguinte, na que están todos os controis modificados con datos erróneos para ver o erro que amosan e tamén como, simultaneamente, o botón de envío fica inhabilitado:

Desta forma acabamos de crear o noso primeiro formulario reactivo en Angular conducido polos datos. Podemos observar que a meirande parte da programación está do lado do TypeScript permitindo desacoplar a vista da lóxica de negocio.

Erros e problemas habituais

Neste apartado veremos algúns dos erros máis habituais que poden xurdir no uso de formularios reactivos en Angular:

- **Erro: o formulario non actualiza os valores no modelo.**
 - Causa: non usar `formControlName` na vista.
 - Solución: asegurarse de empregar o nome do control exacto.
- **Erro: validacións non funcionan.**
 - Causa: non incluír o módulo `ReactiveFormsModule`.

- Solución: importar o módulo no `AppModule` ou no módulo correspondente.
- **Erro: datos incorrectos enviados ao servidor.**
 - Causa: subscrición ao envío sen verificar a validez.
 - Solución: usar `formulario.valid` antes de procesar.
- **Erro: problemas co reset do formulario.**
 - Solución: usar `this.formularioRexistro.reset()` para limpar o estado do formulario.
- **Erro: validación personalizada non é recoñecida.**
 - Causa: non implementar a validación como función ou clase.
 - Solución: revisar que a función devolva un obxecto de tipo `ValidationErrors`.

Cliente HTTP

Angular pon á nosa disposición unha API para comunicacións HTTP que permite a realización de solicitudes e a recepción de respostas dende servidores Web de xeito eficiente e sinxelo. Esta capacidade é imprescindible para a construción de aplicacións Web cliente-servidor, e realízase grazas á funcionalidade contida no módulo `HttpClientModule`.

Para a realización de solicitudes HTTP en Angular emprégase o servizo `HttpClient`, que fornece métodos para as distintas operacións API REST (GET, POST, PUT e DELETE) entre outras. Cada un deses métodos retornará un `Observable` que emitirá o corpo da resposta unha vez completada a solicitude.

Cliente JSON

Neste exemplo recuperaremos un JSON fornecido por un servidor remoto, ao cal accederemos mediante unha petición GET. Con ese fin tomaremos os seguintes pasos:

- Crearemos un servizo (**`solicitud-json`**):

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class SolicitudJsonService {

  constructor(private cliente: HttpClient) { }

  getJSON(url: string) {
    return this.cliente.get(url);
  }
}
```

- No TS da compoñente raíz (**`app.component.ts`**) escribiremos:

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
import { SolicitudJsonService } from './solicitud-json.service';
import { HttpClientModule } from '@angular/common/http';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, HttpClientModule],
  providers: [SolicitudJsonService],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent implements OnInit {
  title = '23-http-client';
  constructor(private servizo: SolicitudJsonService) {}
}
```

```
ngOnInit(): void {
  this.servizo.getJSON('https://jsonplaceholder.typicode.com/users')
    .subscribe(datos => {
      console.log(datos);
    });
}
```

O resultado do JSON recuperado dende o servidor amosarase na consola do navegador.

Exemplo T06E19

Este exemplo realiza unha petición HTTP GET a unha API pública empregando un servizo que encapsula `HttpClient`. O *hook* de inicialización da compoñente chama ao servizo, recibe un `Observable`, subscríbese para obter os datos e gárdalos nun *array* que se amosa na vista mediante `*ngFor`.

O exemplo ten estes obxectivos didácticos:

- Entender o fluxo de traballo con `HttpClient` nun servizo e o retorno como `Observable`.
- Practicar a subscrición en `ngOnInit` para recuperar os datos asíncronos.
- Separar a lóxica de acceso a datos da compoñente mediante un servizo inxectable.
- Empregar a renderización iterativa na vista para mostrar os resultados da petición.

Exemplo T06E20

Este exemplo conecta cun servidor `MockAPI` para xestionar unha lista de usuarios. A compoñente carga os usuarios ao iniciar, permite crear novos mediante enlazado bidireccional, actualizar o rol dun usuario cunha petición `PUT` e eliminalo cunha petición `DELETE`. Toda a comunicación HTTP encapsúlase nun servizo que expón métodos tipados con `Observable`.

Os obxectivos didácticos deste exemplo son:

- Comprender un fluxo CRUD completo (`GET`, `POST`, `PUT`, `DELETE`) contra unha API REST simulada.
- Encapsular chamadas HTTP nun servizo e consumir os `Observables` dende a compoñente.
- Empregar tipado forte con interfaces para datos recibidos/enviados.
- Manexar respostas asíncronas con `subscribe` e actualizar o estado local de forma reactiva.

Aplicación cliente-servidor

Como exercicio final neste curso, procederemos a crear unha aplicación *fullstack*, isto é, unha aplicación que conste tanto de cliente coma de servidor. Así, crearemos un CRUD Angular (o noso cliente) que actuará sobre unha API NodeJS (o noso servidor), e que actuará como un sistema xestor dunha base de datos sobre libros.

Os nosos obxectos libro terán a seguinte estrutura:

- **titulo:** *string*.
- **ISBN:** *string*.
- **autor:** *string*.
- **ano:** *number*.

Servidor

Tal e coma foi indicado na introdución, optamos por empregar NodeJS para desenvolver o noso servidor, xa que xa dispuñamos da tecnoloxía en tanto o motor de Angular funciona sobre a mesma. Para construír o noso servidor tomaremos os seguintes pasos:

- Creamos un directorio para o noso servidor e situámonos no mesmo:

```
mkdir servidor-libros
cd servidor-libros
```

- Creamos un novo proxecto NodeJS:

```
npm init -y
```

- Instalamos as dependencias precisas:

- **Express:** contorna de NodeJS para o desenvolvemento de aplicacións Web.

```
npm install express
```

- **CORS:** módulo que permite limitar o acceso ao noso servidor Web dende distintas ubicacións.

```
npm install cors
```

- **Nodemon (opcional):** permítenos modificar o servidor e que os cambios sexan aplicados *on-the-fly*, sen necesidade de reiniciar o servizo con cada modificación.

```
npm install --save-dev nodemon
```

- Se optamos por instalar **Nodemon**, temos que modificar o ficheiro **package.json** para engadir o correspondente *script* de inicio:

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js"  
},
```

- Creamos o ficheiro principal do servidor (**index.js**) e escribimos o seu código:

```
// INICIALIZACIÓN DO SERVIDOR  
  
// Declaramos os módulos precisos para o funcionamento do servidor  
const express = require("express"); // Express: marco nativo de NodeJS para crear  
servidores Web  
const cors = require("cors"); // CORS: permítenos relaxar a política de seguridade  
do mesmo orixe  
  
// Inicializamos a aplicación express  
const app = express(); // Inicializamos un obxecto de tipo servidor Web  
const port = 3000; // Porto na que escoitará o servidor  
  
// Indicamos ao noso servidor un Middleware para analizar JSONs nas solicitudes  
entrantes  
app.use(express.json());  
  
// Habilitamos CORS para todas as solicitudes (isto permite acceder ao servidor  
dende outros dominios)  
app.use(  
  cors({  
    origin: "*", // Permitir solicitudes de calquera orixe  
  })  
);  
  
// Ruta raíz que devolve unha mensaxe JSON, sirve para saber se o servidor  
arrincou e funciona (por exemplo, dende un navegador)  
app.get("/", (req, res) => {  
  res.json({ message: "Servidor de Libros funcionando!" }); // Respostamos  
mediante un obxecto JSON  
});  
  
// Escoitamos no porto especificado e imprimimos unha mensaxe ao comezar  
app.listen(port, () => {  
  console.log('Servidor escoitando en http://localhost:'+port);  
});  
  
// CORPO DO SERVIDOR  
  
// Esta propiedade, de tipo array de obxectos libro vainos permitir simular unha  
base de datos  
let libros = [  
  // Aquí indicariamos aqueles libros que queiramos que o servidor teña xa dende o  
  arringue  
];
```

```

// Este método define a ruta a través da cal se obterán os datos do servidor (a
que resolverá as peticións de tipo GET)
app.get("/libros", (req, res) => {
    // Devolvemos o array de libros en formato JSON
    if (libros.length > 0) {
        res.json(libros);
    } else {
        res.status(404).json({ message: "Non se atoparon libros." }); // Mensaxe
de erro se non hai libros
    }
});

// Este método define a ruta a través da cal se engadiran novos libros ao servidor
(a que resolverá as peticións de tipo POST)
app.post("/libros", (req, res) => {
    const novoLibro = req.body; // Recuperamos os datos do libro dende o corpo da
solicitud

    // Comprobar se o libro xa existe baseado no ISBN
    if (libros.some((libro) => libro.ISBN === novoLibro.ISBN)) {
        return res.status(409).json({ message: "Xa hai un libro con ese ISBN." });
// Mensaxe de erro se o libro xa existe
    }

    libros.push(novoLibro); // Engadir o novo libro ao array
    res.status(201).json({ message: 'Libro engadido con éxito.' }); // Resposta no
caso de que a solicitude teña éxito
});

// Este método define a ruta a través da cal se actualizarán os libros existentes
no servidor (a que resolverá as peticións de tipo PUT)
app.put('/libros/:ISBN', (req, res) => {
    const ISBN = req.params.ISBN; // Recuperamos o ISBN dende o corpo da
solicitud
    const datosLibroActualizado = req.body; // Obter os datos actualizados do
libro

    // Atopamos o índice do libro procurado no array
    const indiceLibro = libros.findIndex(libro => libro.ISBN === ISBN);
    if (indiceLibro === -1) {
        return res.status(404).json({ message: 'Non hai ningún libro con ese
ISBN.' }); // Mensaxe de erro se non se atopa o libro
    }

    libros[indiceLibro] = { ...libros[indiceLibro], ...datosLibroActualizado }; //
Actualizamos o libro o libro no array
    res.json({ message: 'Libro actualizado con éxito.' }); // Resposta no caso de
que a solicitude teña éxito
});

// Este método define a ruta a través da cal se eliminarán libros do servidor (a
que resolverá as peticións de tipo DELETE)
app.delete('/libros/:ISBN', (req, res) => {
    const ISBN = req.params.ISBN; // Recuperamos o ISBN dende o corpo da
solicitud

    // Atopamos o índice do libro procurado no array
    const indiceLibro = libros.findIndex(libro => libro.ISBN === ISBN);
    if (indiceLibro === -1) {
        return res.status(404).json({ message: 'Non hai ningún libro con ese
ISBN.' }); // Mensaxe de erro se non se atopa o libro
    }

    libros.splice(indiceLibro, 1); // Eliminamos o libro do array
    res.json({ message: 'Libro eliminado con éxito.' }); // Resposta no caso de
que a solicitude teña éxito
});

```

Cliente

O noso cliente combinará aspectos vistos ao longo de toda a nosa aprendizaxe sobre Angular, polo que será con diferenza o exemplo máis completo e complexo visto ao longo do curso. Para construílo precisaremos os seguintes elementos:

- Unha interface: `libro`.
- Un servizo: `libros`.
- Seis compoñentes (ademais da raíz): `buscar`, `consultar`, `engadir`, `listaxe`, `portada` e `xestionar`.

O ficheiro de rutas

No ficheiro `app.routes.ts` engadiremos as rutas que darán acceso ás distintas compoñentes:

```
import { Routes } from '@angular/router';
import { EngadirComponent } from '../engadir/engadir.component';
import { ConsultarComponent } from '../consultar/consultar.component';
import { PortadaComponent } from '../portada/portada.component';
import { XestionarComponent } from '../xestionar/xestionar.component';

export const routes: Routes = [
  { path: 'engadir', title: 'Engadir', component: EngadirComponent },
  { path: 'consultar', title: 'Consultar', component: ConsultarComponent },
  { path: 'xestionar/:ISBN', title: 'Xestionar', component: XestionarComponent },
  { path: '', title: 'Portada', component: PortadaComponent },
];
```

A interface `libro`

Esta interface definirá a estrutura dos obxectos de tipo `libro`, tal e como vimos na introdución do capítulo:

```
export interface Libro {
  titulo: string;
  ISBN: string;
  autor: string;
  ano: number;
}
```

O servizo `libros`

O servizo `libros` será o punto de comunicación do cliente co servidor, actuando de intermediario coa nosa API, é dicir, será dende este servizo dende onde se realizarán as distintas solicitudes API REST cara ao noso servidor.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, map } from 'rxjs';
import { Libro } from '../libro';
```

```

@Injectables({
  providedIn: 'root',
})

export class LibrosService {
  private urlApi = 'http://localhost:3000/libros'; // Enderezo da API (o servidor)

  // Inxectamos un cliente HTTP de Angular que permitirá facer solicitudes ao
  servidor
  constructor(private http: HttpClient) { }

  // Este método recupera todos os libros da API, en forma de Observable de array
  de libros
  obterLibros(): Observable<Libro[]> {
    return this.http.get<Libro[]>(this.urlApi); // Realiza unha solicitude GET
    indicando o tipo desexado e a url da API
  }

  // Este método engade un novo libro á API
  engadirLibro(libro: Libro): Observable<any> {
    return this.http.post(this.urlApi, libro); // Realiza unha solicitude POST,
    enviando os datos do novo libro á API
  }

  // Este método permite actualizar un libro da API
  actualizarLibro(ISBN: string, libro: Libro): Observable<any> {
    return this.http.put(`${this.urlApi}/${ISBN}`, libro); // Realiza unha
    solicitude PUT para actualizar o libro indicando o propio obxecto libro e o seu
    ISBN na ruta
  }

  // Este método permite eliminar un libro da API
  eliminarLibro(ISBN: string): Observable<any> {
    return this.http.delete(`${this.urlApi}/${ISBN}`); // Realiza unha solicitude
    DELETE para eliminar o libro indicando o seu ISBN na ruta
  }

  // Este método permite procurar libros en base a un criterio (algunha das
  propiedades do obxecto libro) e un valor específico
  buscarLibros(criterio: string, valor: string): Observable<Libro[]> {
    return this.obterLibros().pipe(
      map((libros) =>
        libros.filter((libro) => { // Filtra o array de libros baseándose no
          criterio e valor fornecidos
          switch (criterio) { // Comprobamos o criterio escollido para decidir
            como filtrar os libros
            case 'titulo':
              return libro.titulo.toLowerCase().includes(valor.toLowerCase()); //
              Devolvemos os libros cuxo título coincida co indicado
            case 'ISBN':
              return libro.ISBN.includes(valor); // Devolvemos o libro cuxo ISBN
              coincida co indicado
            case 'autor':
              return libro.autor.toLowerCase().includes(valor.toLowerCase()); //
              Devolvemos os libros cuxo autor coincida co indicado
            case 'ano':
              return libro.ano.toString() === valor; // Devolvemos os libros cuxo
              ano coincida co indicado
            default:
              return false; // Se o criterio non coincide con ningunha opción, non
              filtra ningún libro
            }
          })
        )
      );
  }
}

```

A compoñente raíz (app)

Neste exemplo a compoñente raíz será relativamente sinxela, en tanto a práctica totalidade da funcionalidade e estrutura serán asumidas polas outras seis compoñentes do proxecto. Este será o contido dos seus ficheiros:

- CSS:

```
h1 {
  text-align: center;
}
```

- HTML:

```
<h1>Cliente HTTP Libros</h1>
<router-outlet></router-outlet>
```

- TS:

```
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink, RouterOutlet } from '@angular/router';
import { LibrosService } from '../libros.service';
import { HttpClient, HttpClientModule } from '@angular/common/http';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, RouterLink, HttpClientModule],
  providers: [LibrosService],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'cliente HTTP';
  private http = inject(HttpClient);
}
```

A compoñente portada

A aplicación disporá dunha vista principal, representada pola compoñente `portada` (correspondente á ruta por defecto, como podemos ver no ficheiro de rutas), na que aparecerán dúas tarxetas/botóns cos textos «Engadir Libro» e «Consultar Libros» dando acceso ás compoñentes homónimas.

Estes son os seus ficheiros:

- CSS:

```
.container {
  text-align: center;
}
```

```

.tarxetas {
  display: flex;
  justify-content: center;
  gap: 20px;
  margin-top: 20px;
}

.tarxeta {
  border: 1px solid #ddd;
  border-radius: 5px;
  box-shadow: 6px 6px 5px -4px rgba(99, 99, 99, 1);
  font-size: 1.3em;
  padding: 20px;
  width: 150px;
  cursor: pointer;
}

.tarxeta:hover {
  background-color: #f0f0f0;
}

```

- HTML:

```

<div class="container">
  <div class="tarxetas">
    <div class="tarxeta" [routerLink]="['/engadir']">
      <p>Engadir Libro</p>
    </div>
    <div class="tarxeta" [routerLink]="['/consultar']">
      <p>Consultar Libros</p>
    </div>
  </div>
</div>

```

- TS:

```

import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-portada',
  standalone: true,
  imports: [RouterLink],
  templateUrl: './portada.component.html',
  styleUrls: ['./portada.component.css']
})

export class PortadaComponent {}

```

A compoñente engadir

Esta compoñente permitirá, a través dun formulario, crear novos obxectos de tipo libro e, a través do servizo, envialos ao servidor. Este é o contido dos seus distintos ficheiros:

- CSS:

```
.engadir-container {
  max-width: 600px;
  margin: auto;
  padding: 20px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

.engadir-container h2 {
  text-align: center;
}

.engadir-container form div {
  margin-bottom: 10px;
}

.engadir-container form label {
  display: block;
  margin-bottom: 5px;
}

.engadir-container form input[type="text"],
.engadir-container form input[type="number"] {
  width: 100%;
  padding: 8px;
  border: 1px solid #ddd;
  border-radius: 4px;
}

.boton {
  text-align: center;
}

.boton button {
  padding: 10px;
  margin-right: 10px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.boton button:disabled {
  background-color: #ccc;
}

.boton button:not(:disabled) {
  background-color: #007bff;
  color: white;
}

.boton button:nth-child(2) {
  background-color: grey;
  /* Estilo para o botón Cancelar */
}
```

- HTML:

```
<div class="engadir-container">
  <h2>Engadir Novo Libro</h2>
  <form [formGroup]="libroForm" (ngSubmit)="onSubmit()">
    <div>
      <label for="titulo">Titulo:</label>
      <input type="text" id="titulo" formControlName="titulo">
    </div>
    <div>
      <label for="ISBN">ISBN:</label>
      <input type="text" id="ISBN" formControlName="ISBN">
    </div>
    <div>
      <label for="autor">Autor:</label>
      <input type="text" id="autor" formControlName="autor">
    </div>
    <div>
      <label for="ano">Ano:</label>
      <input type="number" id="ano" formControlName="ano">
    </div>
    <div class="botons">
      <button type="submit" [disabled]="!libroForm.valid">Engadir
Libro</button>
      <button type="button" (click)="cancelar()">Cancelar</button>
    </div>
  </form>
</div>
```

- TS:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from
'@angular/forms';
import { LibrosService } from '../libros.service';
import { Router } from '@angular/router';

@Component({
  selector: 'app-engadir',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './engadir.component.html',
  styleUrls: ['./engadir.component.css'],
})

export class EngadirComponent {
  libroForm: FormGroup;

  constructor(private fb: FormBuilder, private librosService: LibrosService,
private router: Router) {
    this.libroForm = this.fb.group({
      titulo: ['', Validators.required],
      ISBN: ['', Validators.required],
      autor: ['', Validators.required],
      ano: ['', Validators.required],
    });
  }
}
```

```

onSubmit() {
  // Compróbase que o formulario sexa válido (é dicir, que o sexan todos os seus
  campos)
  if (this.libroForm.valid) {
    // Chamamos ao método 'engadirLibro' do servizo pasándolle o obxecto Libro
    formado polos datos do formulario. Este método devolverá un Observable coa
    resposta HTTP dende o servidor.
    this.librosService.engadirLibro(this.libroForm.value).subscribe((response)
=> {
      console.log('Libro engadido', response);
      this.libroForm.reset(); // Limpamos o formulario
      this.router.navigate(['/']); // Retornamos á páxina de portada
    });
  }
}

cancelar() {
  this.libroForm.reset(); // Limpamos o formulario
  this.router.navigate(['/']); // Retornamos á páxina de portada
}
}

```

A compoñente consultar

A compoñente consultar amosará as compoñentes buscar e listaxe e actuará coma intermediaria do fluxo de información entre ambas. Así, constará dunha propiedade de tipo *array* de libros que recibirá a través de `@Output()` dende a compoñente buscar e que pasará a través de `@Input()` á compoñente listaxe.

Os seus ficheiros conterán o seguinte:

- CSS:

```

.contedor-consultas {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
}

```

- HTML:

```

<div class="contedor-consultas">
  <app-buscar (resultadosBusca)="manexarResultados($event)"></app-buscar>
  <!-- Só amosamos a compoñente 'listaxe' no caso de que haxa libros no array-->
  <app-listaxe [resultadosBusca]="libros" *ngIf="libros.length != 0"></app-
listaxe>
</div>

```

- TS:

```

import { Component } from '@angular/core';
import { BuscarComponent } from '../buscar/buscar.component';
import { ListaxeComponent } from '../listaxe/listaxe.component';
import { XestionarComponent } from '../xestionar/xestionar.component';
import { Libro } from '../libro';
import { CommonModule } from '@angular/common';

```

```

@Component({
  selector: 'app-consultar',
  standalone: true,
  imports: [CommonModule, BuscarComponent, ListaxeComponent, XestionarComponent],
  templateUrl: './consultar.component.html',
  styleUrls: ['./consultar.component.css']
})

// Esta compoñente actúa como nai e intermediaria entre as compoñentes 'buscar' e
// 'listaxe', de xeito que recibe un array de libros dende 'buscar' (onde está
// decorado con @Output) e envía cara 'listaxe' (onde está decorado con @Input)
export class ConsultarComponent {
  libros: Libro[];

  constructor() {
    this.libros = [];
  }

  manexarResultados(resultados: Libro[]) {
    this.libros = resultados;
  }
}

```

A compoñente buscar

A compoñente `buscar` disporá dun campo de texto, un selector e un botón de procura; de xeito que permita procurar libros en base a distintos criterios. Terá unha propiedade decorada con `@Output()`, de xeito que o resultado da procura poida ser pasado á súa compoñente nai (`consultar`).

Vexamos a súa estrutura:

- CSS:

```

.buscar-container form {
  max-width: 800px;
  min-width: 600px;
  margin: auto;
  padding: 20px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  display: flex;
  justify-content: space-between;
  gap: 10px;
}

.buscar-container input,
.buscar-container select {
  flex-grow: 1;
}

.buscar-container button {
  cursor: pointer;
}

```

- HTML:

```
<div class="buscar-container">
  <form [formGroup]="buscarForm" (ngSubmit)="realizarBusca()">
    <input type="text" formControlName="busqueda" placeholder="Buscar..."
name="buscar">
    <select formControlName="criterio" name="criterio">
      <option value="titulo">Título</option>
      <option value="ISBN">ISBN</option>
      <option value="autor">Autor</option>
      <option value="ano">Ano</option>
    </select>
    <button type="submit">🔍</button>
  </form>
</div>
```

- TS:

```
import { Component, EventEmitter, Output } from '@angular/core';
import { FormBuilder, FormGroup, ReactiveFormsModule } from '@angular/forms';
import { LibrosService } from '../libros.service';
import { Libro } from '../libro';

@Component({
  selector: 'app-buscar',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './buscar.component.html',
  styleUrls: ['./buscar.component.css']
})

export class BuscarComponent {
  buscarForm: FormGroup;
  @Output() resultadosBusca = new EventEmitter<Libro[]>();

  constructor(private fb: FormBuilder, private librosService: LibrosService) {
    this.buscarForm = this.fb.group({
      criterio: ['titulo'], // Valor por defecto para o selector
      busqueda: ['']
    });
  }

  realizarBusca() {
    const valor = this.buscarForm.value; // Recuperamos os datos do formulario

    // Chamamos ao servizo para realizar a busca pasándolle o criterio (título,
    ISBN, autor ou ano) e o valor a procurar. Dado que o método 'buscaLibros' do
    servizo devolve un Observable, suscribímonos e manexamos os resultados recibidos.
    this.librosService.buscarLibros(valor.criterio,
    valor.busqueda).subscribe(resultados => {
      this.resultadosBusca.emit(resultados); // Emítense os resultados da procura
      cara á compoñente nai
    });
  }
}
```

A compoñente **listaxe**

Esta compoñente só será cargada cando haxa resultados procedentes da compoñente `buscar` e, como mencionamos anteriormente, recibirá un *array* de libros dende a compoñente `consultar` o cal amosará en forma de táboa de libros clicable.

Este é o seu código:

- CSS:

```
.listaxe-contedor {
  margin: 20px;
  overflow-x: auto;
  max-width: 800px;
  min-width: 600px;
  padding: 20px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  display: flex;
  justify-content: space-between;
  gap: 10px;
}

table {
  width: 100%;
  border-collapse: collapse;
  cursor: pointer;
}

th,
td {
  padding: 8px;
  text-align: left;
  border-bottom: 1px solid #ddd;
}

tr:hover {
  background-color: #f5f5f5;
}
```

- HTML:

```
<div class="listaxe-contedor">
  <table>
    <thead>
      <tr>
        <th>Titulo</th>
        <th>ISBN</th>
        <th>Autor</th>
        <th>Ano</th>
      </tr>
    </thead>
    <tbody>
      <!-- Construímos o corpo da táboa a partires dos datos do array. Cada
      fila será clicable, de xeito que chamará ao método "seleccionarLibro" pasándolle o
      obxecto libro correspondente á fila seleccionada -->
      <tr *ngFor="let libro of resultadosBusca"
      (click)="seleccionarLibro(libro)">
        <td>{{ libro.titulo }}</td>
        <td>{{ libro.ISBN }}</td>
        <td>{{ libro.autor }}</td>
        <td>{{ libro.ano }}</td>
      </tr>
    </tbody>
  </table>
</div>
```

- TS:

```
import { Component, Input } from '@angular/core';
import { Libro } from '../libro';
import { CommonModule } from '@angular/common';
import { Router } from '@angular/router';

@Component({
  selector: 'app-listaxe',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './listaxe.component.html',
  styleUrls: ['./listaxe.component.css']
})

export class ListaxeComponent {
  @Input() resultadosBusca: Libro[];

  constructor(private router: Router) {
    this.resultadosBusca = [];
  }

  // Este método recibe o libro correspondente á fila da táboa sobre a cal fíxose
  clic e navega á compoñente 'xestionar' pasándolle á ruta o ISBN como parámetro
  seleccionarLibro(libro: Libro): void {
    this.router.navigate(['/xestionar', libro.ISBN]);
  }
}
```

A compoñente `xestionar`

A compoñente `xestionar` recibirá a través da súa ruta un parámetro co ISBN do libro escollido na compoñente `listaxe`. Partindo dese parámetro recuperará o resto de datos do libro a través do servizo e encherá un formulario dende o cal poderemos tanto modificar o libro como eliminalo.

Estes son os seus ficheiros:

- CSS:

```
.xestionar-container {
  max-width: 600px;
  margin: auto;
  padding: 20px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

.xestionar-container h2 {
  text-align: center;
}

.xestionar-container form div {
  margin-bottom: 10px;
}

.xestionar-container form label {
  display: block;
  margin-bottom: 5px;
}
```

```

.xestionar-container form input[type="text"],
.xestionar-container form input[type="number"] {
    width: 100%;
    padding: 8px;
    border: 1px solid #ddd;
    border-radius: 4px;
}

.boton {
    text-align: center;
}

.boton button {
    padding: 10px;
    margin-right: 10px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}

.boton button:disabled {
    background-color: #ccc;
}

.boton button:not(:disabled) {
    background-color: #007bff;
    color: white;
}

.boton button:nth-child(2) {
    background-color: grey;
    /* Estilo para o botón Cancelar */
}

```

- HTML:

```

<div class="xestionar-container">
  <h2>Xestionar Libro</h2>
  <form [formGroup]="libroForm" (ngSubmit)="onSubmit()">
    <div>
      <label for="titulo">Título:</label>
      <input type="text" id="titulo" formControlName="titulo">
    </div>
    <div>
      <label for="ISBN">ISBN:</label>
      <input type="text" id="ISBN" formControlName="ISBN">
    </div>
    <div>
      <label for="autor">Autor:</label>
      <input type="text" id="autor" formControlName="autor">
    </div>
    <div>
      <label for="ano">Año:</label>
      <input type="number" id="ano" formControlName="ano">
    </div>
    <div class="boton">
      <button type="submit">Modificar</button>
      <button type="button" (click)="eliminar()">Eliminar</button>
      <button type="button" (click)="cancelar()">Cancelar</button>
    </div>
  </form>
</div>

```

- TS:

```
import { Component, OnInit } from '@angular/core';
import { Libro } from '../libro';
import { ActivatedRoute, Router } from '@angular/router';
import { LibrosService } from '../libros.service';
import {
  FormBuilder,
  FormGroup,
  ReactiveFormsModule,
  Validators,
} from '@angular/forms';

@Component({
  selector: 'app-xestionar',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './xestionar.component.html',
  styleUrls: ['./xestionar.component.css'],
})

export class XestionarComponent implements OnInit {
  libroForm: FormGroup; // Definimos o noso formulario mediante un grupo
  libro: Libro; // Esta propiedade almacenará o obxecto libro a xestionar
  ISBNOrixinal: string; // Almacena o ISBN orixinal para usalo nas operacións de
  actualización e eliminación

  // Construtor para inxectar dependencias
  constructor(
    private ruta: ActivatedRoute,
    private librosService: LibrosService,
    private construtorForm: FormBuilder,
    private enrutador: Router
  ) {
    // Inicializa o formulario con campos vacíos e validacións
    this.libroForm = this.construtorForm.group({
      titulo: ['', Validators.required],
      ISBN: [{ value: '', disabled: true }, Validators.required],
      autor: ['', Validators.required],
      ano: ['', Validators.required],
    });
    this.ISBNOrixinal = '';
  }

  ngOnInit() {
    // Accedemos aos parámetros da ruta
    this.ruta.params.subscribe((params) => {
      const ISBN = params['ISBN']; // Recuperamos o ISBN dende os parámetros

      // Solicitamos todos os libros ao servizo e procuramos o que coincida co
      ISBN recibido como parámetro na ruta
      this.librosService.obterLibros().subscribe((libros) => {
        this.libro = libros.find((libro) => libro.ISBN === ISBN) as Libro;

        // Asignamos os valores do libro atopado ao formulario
        this.libroForm.patchValue(this.libro);

        this.ISBNOrixinal = this.libro.ISBN; // Garda o ISBN orixinal
      });
    });
  }
}
```

```

// Este método permite actualizar un libro existente no servidor
onSubmit() {
  // Se os datos do formulario son válidos
  if (this.libroForm.valid) {
    this.librosService.actualizarLibro(this.ISBNOrixinal,
this.libroForm.value).subscribe(() => {
      console.log('Libro actualizado');
      this.enrutador.navigate(['/']); // Retornamos á páxina de portada
    });
  }
}

// Este método permite eliminar un libro do servidor
eliminar() {
  this.librosService.eliminarLibro(this.ISBNOrixinal).subscribe(() => {
    console.log('Libro eliminado');
    this.enrutador.navigate(['/']); // Retornamos á páxina de portada
  });
}

// Este método permite cancelar a operación
cancelar() {
  this.enrutador.navigate(['/']); // Retornamos á páxina de portada
}
}

```

Exemplo T06E21

Neste exemplo constrúese unha pequena aplicación Angular que se comunica cun *backend* REST para realizar operacións CRUD. Emprégase un servizo inxectable que encapsula as chamadas HTTP, compoñentes que amosan e actualizan os datos, e observables para reaccionar ás respostas do servidor e refrescar a vista sen recargar a páxina.

A través deste exemplo perseguimos os seguintes obxectivos didácticos:

- Entender a configuración e uso de `HttpClient` para chamadas GET, POST, PUT e DELETE.
- Separar responsabilidades creando un servizo que encapsule a lóxica de acceso a datos.
- Empregar interfaces TypeScript para tipar os modelos transferidos entre cliente e servidor.
- Subscribirse a observables e actualizar o estado da UI de forma reactiva.
- Xestionar estados básicos da petición (carga, éxito, erro) con *feedback* na interface.
- Integrar formularios (*template-driven* ou reactivos) para crear e modificar rexistros.
- Consolidar o fluxo completo cliente → servizo → API e o retorno de datos cara á compoñente.

Exemplos adicionais

Neste apartado engadiremos unha serie de exemplos útiles de aspectos que poderemos integrar nas nosas aplicacións.

Exemplo T06E22

Neste exemplo cárganse datos dun JSON local e amósanse na vista. Emprégase a importación directa do ficheiro JSON e a iteración na plantilla para listar os elementos, evitando chamadas HTTP cando a información é estática e forma parte do *bundle*.

Este exemplo presenta os seguintes obxectivos didácticos:

- Comprender como importar un JSON local como módulo en Angular/TypeScript.
- Coñecer o papel de `resolveJsonModule` para permitir importacións de JSON.
- Entender por que a carga produce un *export* por defecto e como acceder aos datos resultantes.
- Practicar a iteración e presentación de datos coa directiva estrutural correspondente.
- Diferenciar entre datos estáticos empaquetados e datos remotos que requiren `HttpClient`.

Exemplo T06E23

Este exemplo mostra como ler e escribir en WebStorage (`localStorage`) nunha aplicación Angular en modo *standalone* garantindo que o acceso só se realiza no navegador. Emprega dúas estratexias equivalentes para evitar erros en contornos sen DOM: comprobación de plataforma con `isPlatformBrowser(PLATFORM_ID)` e verificación `typeof localStorage !== 'undefined'`. O valor lido amósase na vista mediante interpolación de *strings*.

O exemplo persegue os seguintes obxectivos didácticos:

- Comprender que WebStorage só está dispoñible no navegador e pode fallar en SSR/tests.
- Aplicar patróns seguros de acceso: `isPlatformBrowser` e comprobación de existencia de `localStorage`.
- Practicar a escritura e lectura de pares clave/valor con `localStorage` e a súa visualización na plantilla.

Exemplo T06E24

Neste exemplo créase un control de formulario a medida (app-pasoapaso) que incrementa/decrementa un valor numérico e intégrase nun formulario reactivo mediante `ControlValueAccessor`. A compoñente fornece o seu valor e estado ao formulario a través de `NG_VALUE_ACCESSOR`, implementando `writeValue`, `registerOnChange`, `registerOnTouched` e `setDisabledState`. O control úsase con `formControlName` nun `FormGroup` xunto con outros campos, permitindo gardar o conxunto de datos do formulario.

O exemplo persegue estes obxectivos didácticos:

- Entender o patrón `ControlValueAccessor` para conectar un control personalizado co sistema de formularios reactivos.
- Aprender a expor cambios e eventos de «tocado» ao formulario usando `registerOnChange` e `registerOnTouched`.
- Aplicar `writeValue` e `setDisabledState` para sincronizar valor e estado (habilitado/deshabilitado) dende fóra do control.
- Configurar `NG_VALUE_ACCESSOR` cun `provider` e `forwardRef` nunha compoñente *standalone*.
- Integrar o control personalizado nun `FormGroup` e enviar os datos do formulario completo.

Exemplo T06E25

Este exemplo ilustra o ciclo de vida das compoñentes en Angular. A compoñente filla rexistra, mediante un `signal`, os eventos que se van disparando nos principais *hooks* (`ngOnInit`, `ngOnDestroy`, `ngOnChanges`, `ngDoCheck`, `ngAfterContentInit`, `ngAfterContentChecked`, `ngAfterViewInit`, `ngAfterViewChecked`). A compoñente nai permite provocar cambios de entrada e montar/desmontar a compoñente para observar a execución de cada *hook* en orde.

Este exemplo ten os seguintes obxectivos didácticos:

- Comprender a finalidade e a orde de execución dos *hooks* do ciclo de vida dunha compoñente.
- Observar como os cambios en entradas `@Input` e no estado interno disparan distintos *hooks*.
- Practicar o montado e desmontado dunha compoñente para provocar `ngOnInit` e `ngOnDestroy`.
- Empregar *signals* para rexistrar eventos de forma reactiva e visualizalos na vista.
- Diferenciar cambios estruturais na árbore de vistas (`*ngIf`) de cambios de datos internos.

Exemplo T06E26

Este exemplo mostra o uso de *pipes* predefinidos en Angular para transformar e formatear datos na vista. A compoñente principal define exemplos de datos (texto, data, número, porcentaxe e importe) e aplica os *pipes* `uppercase`, `lowercase`, `titlecase`, `date: 'HH:mm dd-MM-yyyy'`, `number: '1.2-2'`, `percent` e `currency: 'EUR': 'symbol': '1.2-2'` para mostrar distintos formatos.

O exemplo posúe os seguintes obxectivos didácticos:

- Comprender o uso dos *pipes* predefinidos para transformar datos en *templates*.
- Aprender a formatear *strings*, datas, números, porcentaxes e moedas.
- Practicar o paso de argumentos aos *pipes* (patróns de data, precisión decimal, símbolo de moeda).
- Ver como combinar e encadear *pipes* na vista.
- Observar o impacto da localización e dos patróns de formato en saídas numéricas e de data.

Exemplo T06E27

Este exemplo presenta a creación dun *pipe* personalizado chamado `truncarTexto` que acurta cadeas sen cortar palabras polo medio, procurando o último espazo dentro dun límite de caracteres e engadindo puntos suspensivos. O *pipe* é *standalone*, recibe un parámetro de lonxitude cun valor por defecto de 200 e úsase na compoñente principal para demostrar truncados a 100, 150 e 200 caracteres, ademais de amosar o texto orixinal.

Obxectivos didácticos deste exemplo:

- Comprender como crear e rexistrar un *pipe* personalizado en Angular.
- Aprender a implementar a lóxica de truncado que evita cortar palabras pola metade.
- Practicar o uso de parámetros en *pipes* e valores por defecto (lonxitude por defecto 200).
- Ver como declarar e usar *pipes standalone* e importalos nunha compoñente *standalone*.
- Observar o efecto do pipe aplicándoos a diferentes límites (100, 150, 200) e comparar co texto orixinal.

Exemplo T06E28

Este exemplo mostra unha directiva personalizada `appResaltar` que aplica unha cor de fondo ao elemento cando o punteiro do rato entra no seu alcance e restaura a cor orixinal ao saír. A directiva acepta un parámetro `@Input` chamado `fondo` (valor por defecto `yellow`) e emprega `ElementRef` e `Renderer2` para modificar estilos de forma segura. A vista de proba aplica a directiva en elementos distintos (`p`, `button`, `div`) para demostrar o efecto.

Obxectivos didácticos do exemplo:

- Entender como crear e usar directivas personalizadas en Angular.
- Aprender a usar `ElementRef` e `Renderer2` para manipulación segura do DOM.
- Practicar a exposición de parámetros vía `@Input` en directivas.
- Observar o uso de `HostListener` para escoitar eventos de *host* como `mouseenter` e `mouseleave`.
- Diferenzar entre aplicar estilos directamente e facelo mediante `Renderer2` por seguridade e compatibilidade.

Exemplo T06E29

Este exemplo introduce un servizo de mensaxería que combina un `BehaviorSubject` para expoñer a última mensaxe e un `ReplaySubject` para gardar as últimas 5 mensaxes, permitindo que compoñentes emisoras envíen mensaxes e compoñentes receptoras as reciban. O servizo tamén usa un `signal` como contador de mensaxes para numerar os envíos; a compoñente emisora chama a `enviarMensaxe(texto)` para difundir a mensaxe, e a compoñente receptora subscríbese ás dúas fontes para actualizar a última mensaxe mostrada e manter un historial.

Este exemplo persegue os seguintes obxectivos didácticos:

- Comprender como usar servizos para compartir estado e eventos entre compoñentes.
- Aprender a empregar `BehaviorSubject` para expoñer o último valor e `ReplaySubject` para gardar un historial limitado.
- Practicar a creación dun punto centralizado de comunicación (pub/sub) entre emisores e receptores.

Ver como combinar APIs reactivas (RxJS) con `signal` para fluxo de datos específico (contador).

Observar subscricións en `ngOnInit` e a actualización de vista desde observables.

Exemplo T06E30

Este exemplo ilustra dous casos de uso de Angular Material: unha compoñente de autocompletado que filtra e suxire días da semana empregando `MatAutocomplete`, `MatFormField` e formularios reactivos; e unha compoñente de paxinación que mostra unha táboa de obxectos `Gato` e utiliza `MatPaginator` para dividir os datos en páxinas.

Obxectivos didácticos deste exemplo:

- Aprender a integrar e empregar compoñentes de Angular Material como `MatAutocomplete`, `MatFormField`, `MatInput`, `MatTable` e `MatPaginator`.
- Entender como conectar controles de formulario (`FormControl`, `ReactiveFormsModule`) con `MatAutocomplete` para filtrar opcións en tempo real.
- Practicar a configurar unha `MatTable` con `MatTableDataSource` e engadir un `MatPaginator` para controlar a paxinación.
- Ver como importar módulos de Angular Material en compoñentes *standalone* e como usalos con `imports`.
- Observar o uso de `AsyncPipe` e `valueChanges` para actualizar listas suxeridas de forma reactiva.

Exemplo T06E31

Este exemplo amosa un formulario reactivo que permite seleccionar un ficheiro mediante un `input` de tipo `file`. A compoñente utiliza `FormBuilder` para crear un `FormGroup` con un control ficheiro, captura cambios no `input` co método `cambioFicheiro` (evento) e almacena o ficheiro seleccionado en datos; ao enviar o formulario, se hai un ficheiro seleccionado, a compoñente imprime o obxecto `File` no console.

Este exemplo ten os seguintes obxectivos didácticos:

- Comprender como integrar `input` de tipo ficheiro en formularios reactivos.
- Aprender a capturar e almacenar un ficheiro seleccionado no `FormGroup`.
- Practicar o uso de `FormBuilder` e `formControlName` con `ReactiveFormsModule`.
- Ver como ler propiedades básicas do ficheiro seleccionado, por exemplo `name`, e preparalo para envío ou procesamento posterior.
- Probar o fluxo completo: seleccionar ficheiro, verificar que aparece o nome na vista e pulsar `Enviar Ficheiro` para comprobar a saída na consola.

Exemplo T06E32

Este exemplo amosa casos de uso de probas unitarias en Angular usando `TestBed`, `Jasmine` e `Karma`. Hai probas para verificar que a compoñente principal se crea correctamente e que o título e a renderización funcionan, e tamén hai unha compoñente `MensaxeComponent` con probas que comprobaban a creación da compoñente, a renderización da mensaxe por defecto e a actualización da mensaxe ao cambiar a propiedade.

O exemplo pretende acadar os seguintes obxectivos didácticos:

- Comprender como configurar e empregar TestBed para crear bancos de proba de compoñentes.
- Aprender a escribir probas con Jasmine (suites describe, casos *it*, expectativas expect).
- Practicar a inspección da vista renderizada mediante ComponentFixture, debugElement e By.css.
- Ver como comprobar a creación da compoñente, a renderización de contido e a reacción aos cambios de propiedade.
- Observar a configuración típica de probas unitarias en proxectos Angular (TestBed + Jasmine + Karma).

Exemplo T06E33

Este exemplo combina un CRUD de tarefas no *front* con probas unitarias deseñadas para verificar tanto a lóxica do servizo como o comportamento das compoñentes que interactúan co servizo. O servizo `TarefasService` utiliza `signal` para manter un estado reactivo e expón métodos para obter tarefas, engadir tarefas e marcar tarefas como completadas; a compoñente `lista-tarefas` mostra as tarefas e actualízase en resposta aos cambios do servizo, e a compoñente `nova-tarefa` emite un evento co título para engadir novas tarefas. As probas usan TestBed, Jasmine e Karma para comprobar a funcionalidade do servizo, a emisión de eventos desde a compoñente, e que a interface reflicte correctamente as operacións CRUD.

O exemplo persegue os seguintes obxectivos didácticos:

- Comprobar que `TarefasService` inicializa o estado correctamente e que `obterTarefas()` devolve as tarefas esperadas.
- Verificar que `engadirTarefa(novaTarefa)` engade correctamente unha nova tarefa e que o `signal` reflicte o cambio.
- Probar que `completarTarefa(id)` marca a tarefa correcta como completada e non altera outras tarefas.
- Asegurar que a compoñente `nova-tarefa` emite o evento `tarefaEngadida` co título correcto e que limpa o campo de entrada.
- Probar que a compoñente `lista-tarefas` recibe as tarefas do servizo e renderiza a lista correctamente.
- Comprobar, usando `ComponentFixture` e `TestBed`, que ao emitir o evento desde `nova-tarefa` a `lista-tarefas` actualiza a vista e aparece a nova tarefa.
- Ver que o botón de completar queda deshabilitado cando a tarefa xa está completada.
- Empregar `spies` ou `mocks` para illar o servizo nas probas de compoñente cando sexa necesario.

- Probar casos de entrada non válidos (por exemplo título baleiro) para garantir que non se engaden tarefas inválidas.

Exemplo T06E34

Este exemplo mostra como interactuar cunha API HTTP desde un servizo Angular e como escribir probas unitarias para ese servizo enfocadas nas chamadas HTTP; o servizo usa `HttpClient` para obter e engadir tarefas; as probas empregan `HttpTestingController` para interceptar e simular respostas, permitindo verificar métodos, URL e corpos das peticións e comprobar o comportamento dos subscritores e das compoñentes que consumen o servizo.

Obxectivos didácticos perseguidos co exemplo:

- Comprobar que `TarefasService.obterTarefas()` fai unha chamada GET á URL correcta e que a subscrición recibe os datos simulados esperados.
- Verificar que `TarefasService.engadirTarefa()` fai unha chamada POST á URL correcta co corpo adecuado e que a resposta se propaga ao subscritor.
- Aprender a configurar `TestBed` con `provideHttpClientTesting()` e a obter `HttpTestingController` para interceptar e simular peticións HTTP.
- Practicar a simulación de respostas con `httpMock.expectOne(...).flush(...)` e a verificación do método HTTP e do corpo da petición.
- Probar a xestión de erros simulando respostas con códigos de erro e verificar o comportamento do subscritor fronte a eses erros.
- Comprobar a integración a nivel de compoñente substituíndo o servizo por un *mock* e verificar que a compoñente procesa e amosa correctamente os datos obtidos.
- Practicar o uso de observables e técnicas para probar asincronía en probas unitarias.

Bibliografía

- Angular – Seguridad, protegiendo vistas con guards: <https://codingpotions.com/angular-seguridad/>
- «El gran libro de Angular». (2018). Boada, M. e Gómez, J.A. Marcombo (Barcelona).
- Guards - Curso de Angular (carheco – GitHub): <https://github.com/carherco/curso-angular/blob/master/docs/guards.md>
- Página oficial de **Angular**: <https://angular.io/>

ÍNDICE

INTERFACES	1
TIPOS DE INTERFACES	1
CREACIÓN DE INTERFACES	1
USO DE INTERFACES	3
ERROS E PROBLEMAS HABITUAIS.....	4
CLASES.....	6
CREACIÓN DE CLASES	6
USO DE CLASES	8
ERROS E PROBLEMAS HABITUAIS.....	9
MÓDULOS	10
CONCEPTOS FUNDAMENTAIS	10
ESTRUTURA DUN MÓDULO.....	11
TIPOS DE MÓDULOS	11
CREACIÓN DE MÓDULOS	12
CARGA PREGUICEIRA	13
APLICACIÓNS MODULARES	14
ERROS E PROBLEMAS HABITUAIS.....	16
SERVIZOS.....	17
CREACIÓN DE SERVICIOS	17
INXECTAR E EMPREGAR SERVICIOS.....	17
TIPOS DE SERVICIOS	18
IMPLEMENTANDO SERVICIOS	18
EMPREGANDO SERVICIOS.....	19
ERROS E PROBLEMAS HABITUAIS.....	20
OBSERVABLES.....	21
INSTALANDO AS DEPENDENCIAS	21
EMPREGANDO OBSERVABLES	21
ERROS E PROBLEMAS HABITUAIS.....	26
ANIMACIÓNS.....	27
INSTALANDO AS DEPENDENCIAS	27

EMPREGANDO ANIMACIÓNS	27
EXEMPLO PRÁCTICO: LISTAXE ANIMADA	30
ANIMACIÓNS COMPOSTAS	32
ERROS E PROBLEMAS HABITUAIS.....	32
RUTAS	34
RUTAS SIMPLES.....	34
RUTAS ANIÑADAS.....	38
PASO DE PARÁMETROS	39
RUTAS CON <i>GUARDS</i>	43
CARGA PREGUICEIRA DE RUTAS	48
ERROS E PROBLEMAS HABITUAIS.....	48
FORMULARIOS REACTIVOS	50
CREACIÓN DE FORMULARIOS REACTIVOS	50
VALIDACIÓN DE FORMULARIOS REACTIVOS	53
ERROS E PROBLEMAS HABITUAIS.....	59
CLIENTE HTTP	61
CLIENTE JSON.....	61
APLICACIÓN CLIENTE-SERVIDOR.....	63
SERVIDOR	63
CLIENTE.....	66
EXEMPLOS ADICIONAIS.....	80
BIBLIOGRAFÍA.....	87